ML Experiment Detective: Complete Learning Guide

From Fashion-MNIST to semantic retrieval, RAG, and a local tool-using agent

A beginner-first walkthrough of the working repository, its saved measurements, and the exact question-to-answer path.

44 chapters / 22 vector figures / 4 appendices

Study edition - 3 October 2026

Contents

Chapter 1 - What are we trying to build? 5

Chapter 2 - Our first ML task: Fashion-MNIST 6

Chapter 3 - Train, validation, and test 7

Chapter 4 - Randomness and seeds 8

Chapter 5 - Batches and epochs 9

Chapter 6 - Our simple neural network 10

Chapter 7 - Training the model 11

Chapter 8 - What is an ML experiment? 12

Chapter 9 - Saving experiments as JSON 13

Chapter 10 - Experiment notes 14

Chapter 11 - Our first search: keyword matching 15

Chapter 12 - What is an embedding? 16

Chapter 13 - Tokenization and the embedding model 17

Chapter 14 - Cosine similarity 18

Chapter 15 - Semantic search 19

Chapter 16 - Retrieval evaluation 20

Chapter 17 - Structured filters and semantic search 21

Chapter 18 - What is RAG? 22

Chapter 19 - Evidence packets 23

Chapter 20 - What is an LLM? 24

Chapter 21 - What is Ollama? 25

Chapter 22 - Why not just ask Ollama everything? 26

Chapter 23 - Tool calling 27

Chapter 24 - Tool-calling message flow 28

Chapter 25 - What is an agent? 29

Chapter 26 - The agent loop in our project 30

Chapter 27 - Fixed RAG pipeline versus agentic retrieval 31

Chapter 28 - RAG as an agent tool 32

Chapter 29 - Complete end-to-end trace 33

Chapter 30 - Complete architecture 35

Chapter 31 - Responsibility separation 37

Chapter 32 - APP POLICY and APP FILTER 38

Chapter 33 - Local execution 39

Chapter 34 - Project file walkthrough 40

Chapter 35 - Important code walkthrough 41

Chapter 36 - What we learned from failures 44

Chapter 37 - RAG versus Agentic AI 45

Chapter 38 - What this project is not yet 46

Chapter 39 - Where LangGraph would fit later 47

Chapter 40 - Where a vector database would fit later 48

Chapter 41 - Where memory would fit 49

Chapter 42 - Where multi-agent would fit 50

Chapter 43 - Glossary 51

Chapter 44 - Final mental model 53

Appendix A - Complete worked trace 54

Appendix B - Beginner checkpoint answers 55

Appendix C - How to run this checkout 56

Appendix D - A project-specific study order 57

From a first image classifier to retrieval, RAG, and a local tool-using agent. Repository snapshot studied: 3 October 2026. All run values below were checked against the saved JSON records. The worked agent trace is the trace preserved in `docs/TRACE_EXAMPLE.md`.

How to read this guide

Read in order on a first pass. A **run** is one saved training experiment. A **note** is a human-readable description of a run. A **tool** is a Python action that the model may request. Terms are introduced before they are used in depth. Figures show the *flow of information*; their toy numbers are marked as illustrations. The final app's displayed answer is written by Python from evidence, although an earlier project milestone let an LLM generate a checked answer. This distinction matters in Chapters 18, 29, and 37.

The route through the project

Stage	Main question	Where to look
Create evidence	What actually happened during training?	`phase0.py`, `runs/*.json`
Describe it	Which behaviors deserve a short note?	`notes/*.md`
Find it	Which note resembles this question?	`note_retrieval.py`
Inspect it	What do the exact saved numbers show?	`experiment_data.py`
Decide and answer	Which action comes next, and what can be said?	`simple_agent.py`, `app.py`

You can test yourself at five checkpoints (A-E); their answers are collected in Appendix B. Every figure shows a data path or division of responsibility. When a note or score sounds persuasive, follow its run ID back to a JSON record before drawing a conclusion.

Part I - Create evidence: the ML laboratory

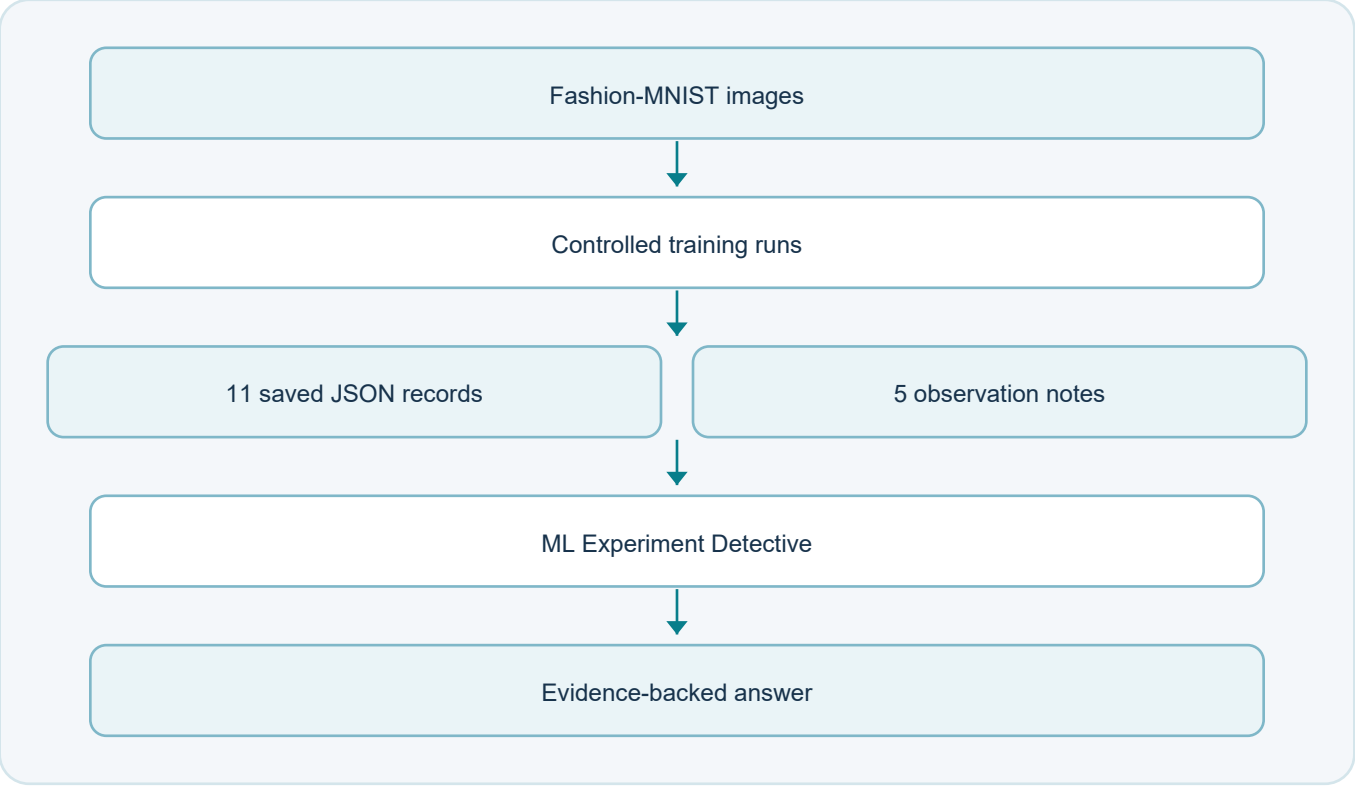
Chapter 1 - What are we trying to build?

An ML engineer trains a model many times: a different learning rate here, another random seed there. A spreadsheet of final scores hides the path each run took. Was one score lower because the validation loss jumped? Which runs used a particular setting? Which older note described similar behavior? Opening every file manually stops being pleasant.

ML Experiment Detective is the local command-line application that helps inspect saved Fashion-MNIST runs, compare their recorded settings and measurements, find notes by meaning, and answer with sources. Think of it as a case desk: the records are evidence, retrieval finds candidate files, and Python reads the measurements.

It can: answer exact run questions, compare two runs, filter recorded settings, search five observation notes semantically, make several tool calls, and show a trace. **It cannot:** prove a cause, train autonomously, search an unlimited research library, remember earlier conversations, or guarantee that a retrieved note is relevant.

Figure 01 - From the image laboratory to the detective



> **Remember this:** Fashion-MNIST creates the experiment history. The Detective studies that history; it is not the image classifier itself.

Chapter 2 - Our first ML task: Fashion-MNIST

A **dataset** is a collection of examples. Each **sample** here is an image of one clothing item. Its **label** is the correct category. Fashion-MNIST has ten categories: T-shirt/top, trouser, pullover, dress, coat, sandal, shirt, sneaker, bag, and ankle boot. The task is to predict one of those ten labels for a new image.

Each image is 28 by 28 pixels. It is grayscale, so each location has one brightness value rather than three color channels. Flattening the grid yields `28 * 28 = 784` values for the simple network. A tensor batch therefore has shape `[batch, 1, 28, 28]`: number of images, one channel, height, width. `ToTensor()` converts images into tensors for PyTorch.

Tiny example: a 2 by 2 grayscale image could be `[[0.0, 0.2], [0.9, 1.0]]`. Flattening gives four numbers. Our real image uses 784. The toy values are illustrative; they are not a saved run.

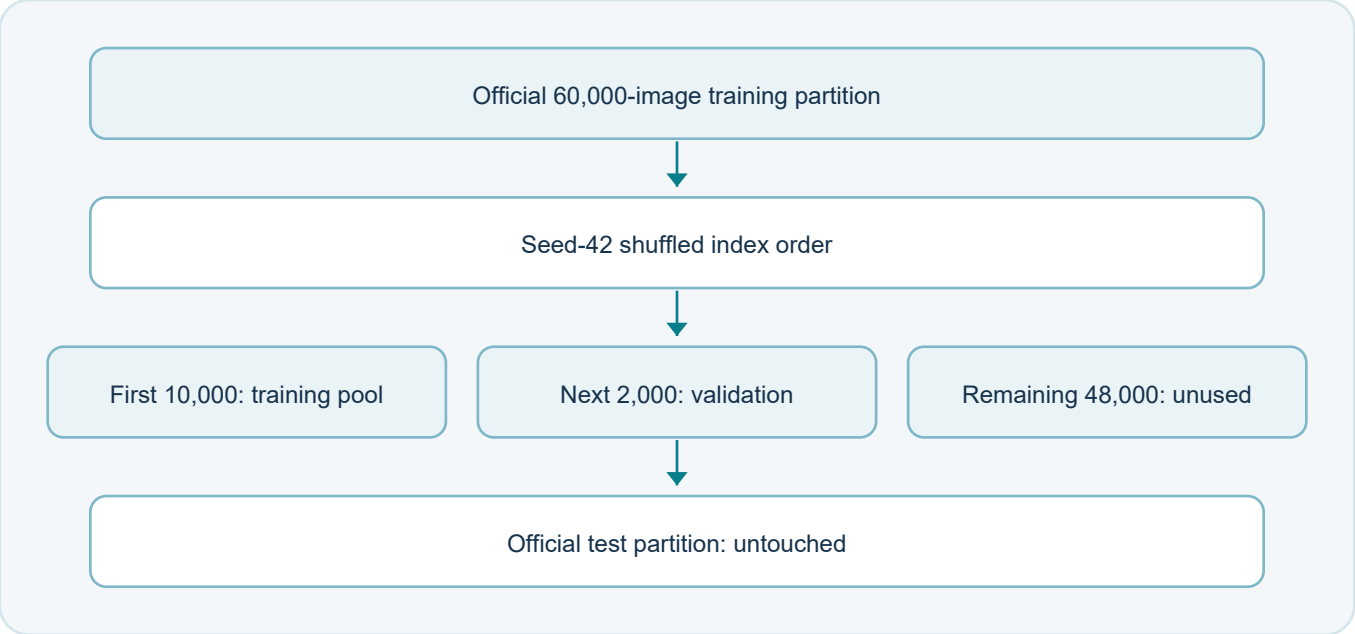
Why this task? It is small enough to make repeated experiments possible while still producing meaningful differences in loss curves. The final app never sees the image pixels when answering a question; it reads the records made after training.

Chapter 3 - Train, validation, and test

The **training set** is the material a model learns from. The **validation set** is a separate practice exam used to observe how well training generalizes. The **test set** is a final exam reserved for a later, unbiased check. If we compare two models on different validation questions, their scores are hard to compare fairly.

In `phase0.py`, a seed-42 permutation of the official 60,000-image *training partition* makes a fixed pool. Indices `0:10000` form the maximum training pool and `10000:12000` form the same 2,000-image validation set. A run with fewer training images takes the beginning of that training pool. The official Fashion-MNIST test partition is not loaded by this script.

Figure 02 - The project's actual split



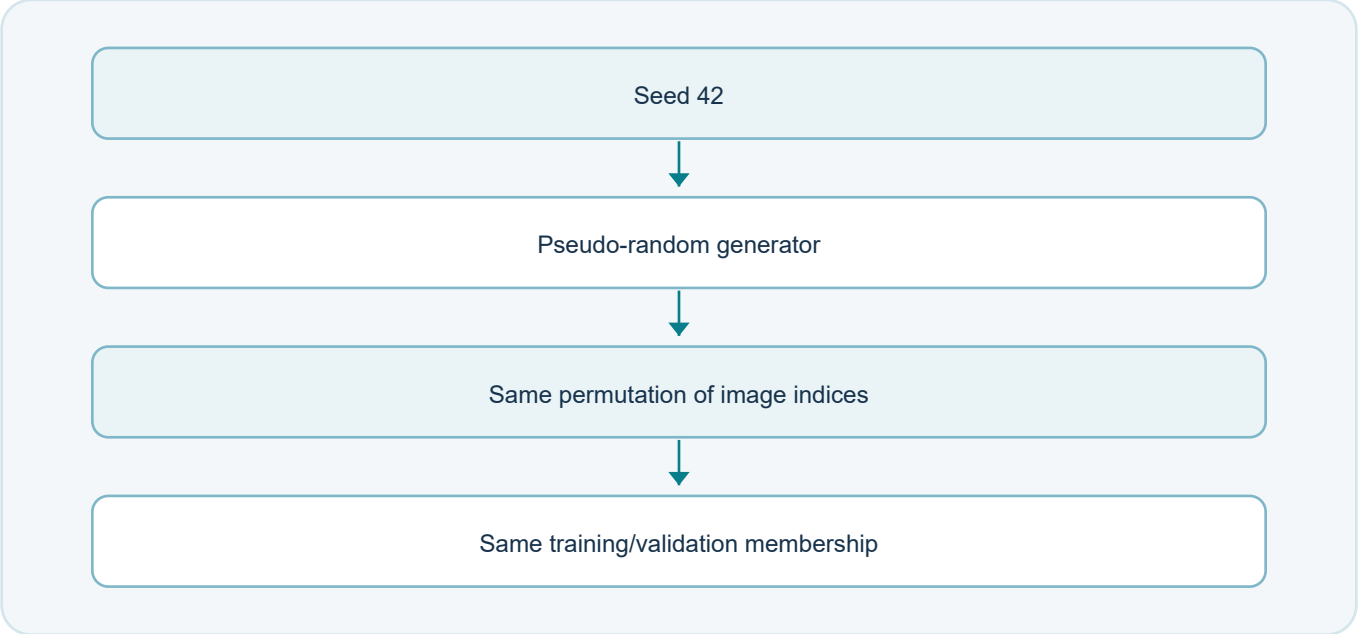
Holding validation fixed means `75.90%` and `71.45%` refer to the same 2,000 examples. It does not by itself make two runs identical in every other respect. If we changed both the learning rate and the validation images, either change could explain a score difference.

Chapter 4 - Randomness and seeds

Computers use a **pseudo-random generator**: the same starting **seed** produces the same sequence, given the same program and environment. A seed is a starting point, not a magic quality score. It controls *which* random outcome is sampled; it does not make a result universally true.

This project records three jobs separately: `split_seed` chooses data indices; `shuffle_seed` orders training batches; `model_seed` sets initial model weights. For `lr_005_seed3` and `lr_01_seed3`, these are 42, 243, and 244 respectively. Their saved settings differ in learning rate, while the recorded seeds agree.

Figure 03 - One seed, reproducible order

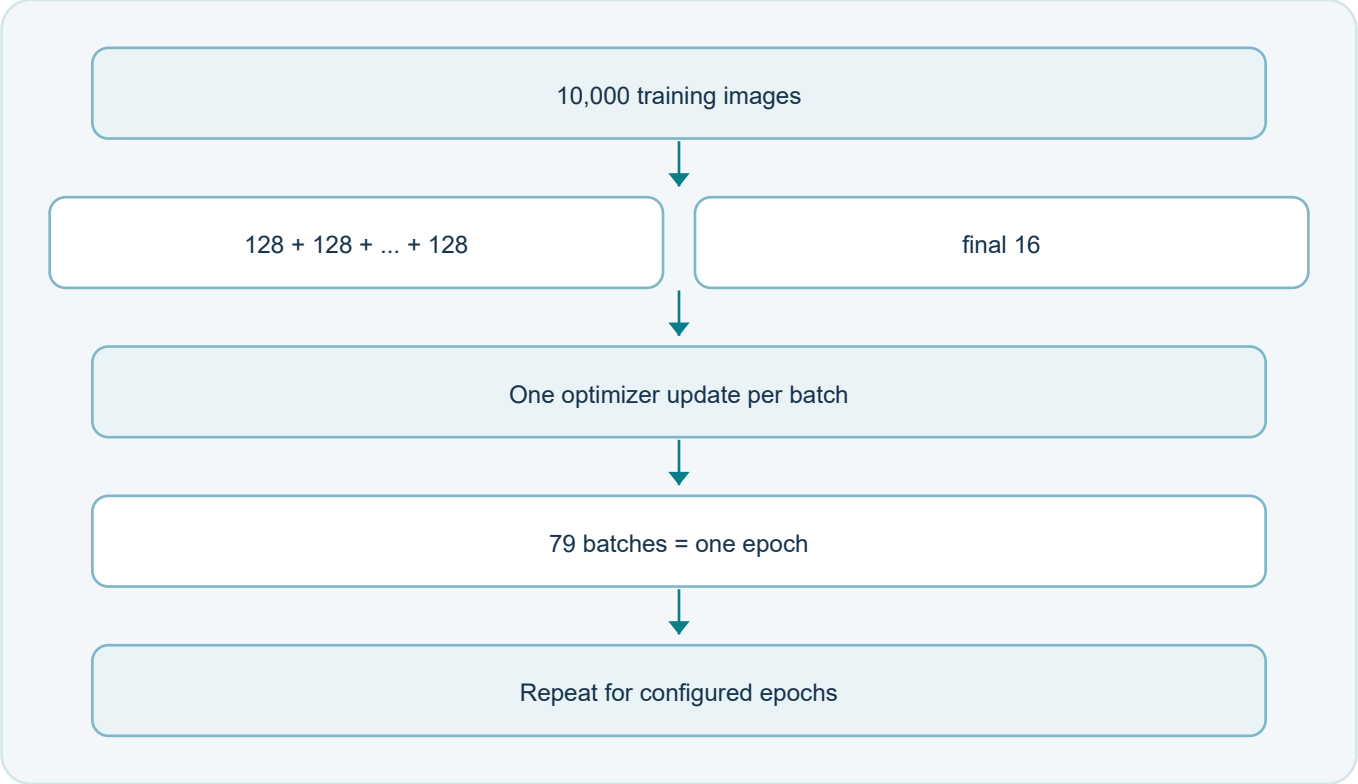


Reproducibility helps isolate a change. Yet a pattern in one seed block can fail in another: the saved seed-2 block has high-learning-rate final accuracy **0.30 percentage points higher**, even though seed-3 has it **4.45 points lower**. That is evidence against a simple universal claim.

Chapter 5 - Batches and epochs

A **batch** is a group of examples used for one weight update. An **epoch** is one full pass through the selected training images. The project uses batch size 128. For 10,000 images, there are 78 full batches of 128 (9,984 images) and one final batch of 16: **79 updates per epoch**. The data are not reduced to 128 images.

Figure 04 - Batches inside an epoch



Smaller batches fit memory and update weights more frequently than one giant batch. The training loader shuffles examples using its own seed; the validation loader does not shuffle because it only measures the model.

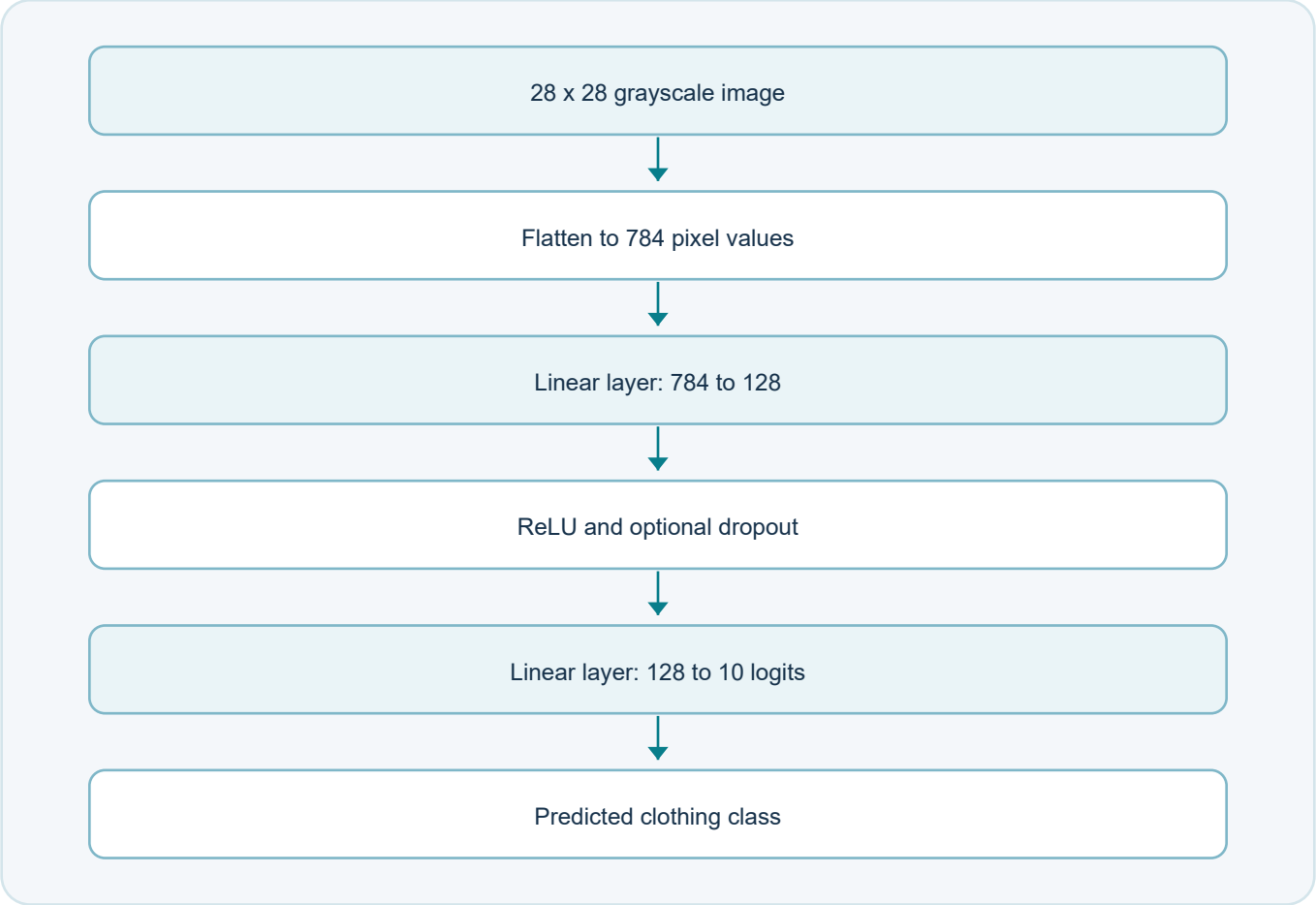
> **You should now understand - checkpoint A:** (A1) Why share one validation set? (A2) What changes when `model_seed` changes? (A3) Does batch size 128 discard the other 9,872 training images?

Chapter 6 - Our simple neural network

A **neuron** combines input numbers with learned **weights**, adds a learned **bias**, then passes the result on. A weight controls how strongly one input influences the calculation. Different initial weights let neurons learn different patterns. The **hidden layer** is an intermediate layer, not a secret part of the program. Here it has 128 units, a design choice that can be changed.

`SimpleMLP` flattens each image to 784 numbers, maps them to 128 hidden values, applies **ReLU** (`max(0, value)`), optionally drops hidden values during training, and maps to ten output **logits**. A logit is an unnormalized class score. The highest logit becomes the predicted label. The output size is ten because there are ten clothing classes.

Figure 05 - The MLP used by phase0.py



What can go wrong? More hidden units do not automatically generalize better. Dropout changes training behavior and must be recorded with the run. A logit is not yet a probability.

Chapter 7 - Training the model

Training repeats a feedback cycle. The network predicts logits; **cross-entropy loss** scores how poorly those logits fit the true labels; **backpropagation** computes which weights contributed to the loss; **SGD** (stochastic gradient descent) updates the weights. The **learning rate** controls update size. `optimizer.zero_grad()` clears old gradients before the new batch's backward pass.

Accuracy is the fraction of correct predicted labels. **Validation loss** and **validation accuracy** are computed on held-out examples after each epoch with `model.eval()` and `torch.no_grad()`. The separate `train_loss_during_epoch` averages losses while weights were changing; `train_loss_after_epoch` measures the final model on training images. They answer different questions.

Training can appear to improve while validation worsens. That may signal overfitting, a temporary fluctuation, or another effect; a single curve is descriptive evidence, not a diagnosis. In `lr_01_seed3`, validation loss rises from `0.6828` at epoch 4 to `0.8109` at epoch 5. The run record shows the change; it does not identify its cause.

Mental model: correct an essay against an answer key, adjust your approach, then test on practice questions you did not study from.

Chapter 8 - What is an ML experiment?

One **run** is one training execution plus its recorded configuration and epoch measurements. A run ID is its stable name. A **hyperparameter** is a setting chosen before training, such as learning rate, batch size, or dropout. A run is useful only when we know what settings and measurements belong together.

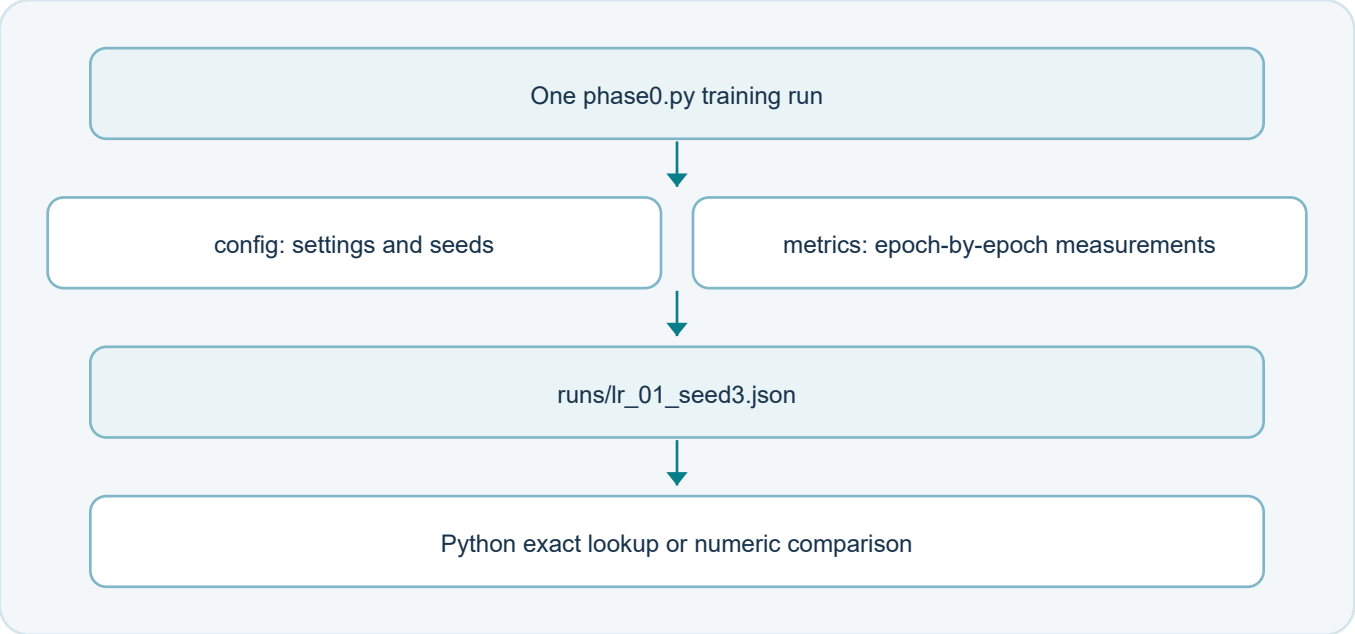
The matched seed-3 pair records learning rate `0.05` in `lr_005_seed3` and `0.1` in `lr_01_seed3`. Both record training size 10,000, validation size 2,000, five epochs, batch size 128, the same seeds, and dropout zero. Their final validation accuracies are `75.90%` and `71.45%`. The observed difference is `-4.45 percentage points` for high minus low learning rate, not `-4.45%` relative change. Repeating a configuration or changing seeds creates another saved run, but repeated records are not automatically independent evidence.

Why do we need many runs? A one-pair comparison can suggest a question. The saved seed-2 pair shows counterevidence to the claim that the higher rate always ends lower. More matched runs would reduce uncertainty, but this project's current learning target is the Detective, not further training.

Chapter 9 - Saving experiments as JSON

JSON is a text format that stores named fields, lists, numbers, and strings. A run file has `config` for setup and `metrics` for the per-epoch history. The saved `runs/lr_01_seed3.json` contains `learning_rate: 0.1`, `split_seed: 42`, `model_seed: 244`, `shuffle_seed: 243`, and five epoch rows. Its final row has `val_accuracy: 0.7145` and `val_loss: 0.810864956855774`; the app displays `71.45%` and `0.8109` after rounding.

Figure 06 - Structured experiment storage



For “Which runs used learning rate 0.1?”, Python checks the exact field `config["learning_rate"] == 0.1`. Embeddings would be an unnecessarily fuzzy answer to an exact database-style question. Missing fields in older JSON records remain **unknown**; absence does not mean zero.

Chapter 10 - Experiment notes

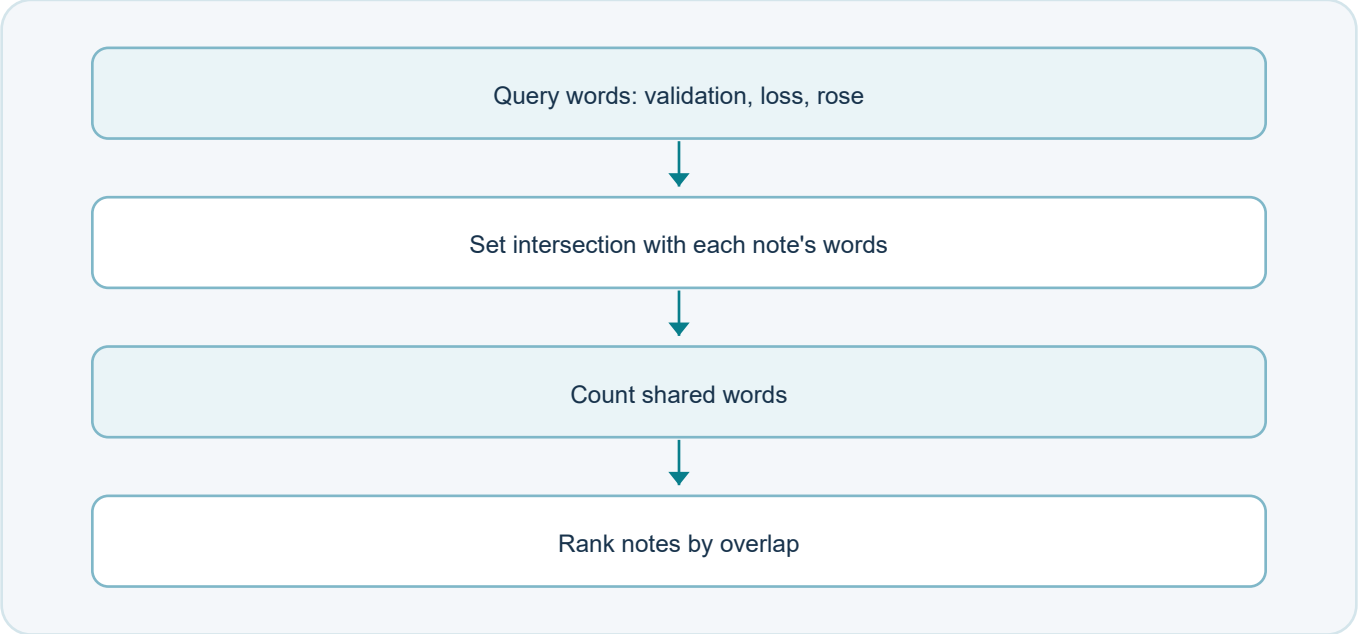
The five files under `notes/` are short **unstructured** descriptions, meaning the interesting behavior is written in sentences rather than in a fixed field. `notes/lr_01_seed2.md` says validation loss rose from `0.7136` (epoch 3) to `0.8142` (epoch 4), then fell to `0.6793` (epoch 5). `notes/baseline_gap_001.md` describes a different rise and fall.

Each note is a paraphrase of its primary run record, **not another independent measurement**. The `small_dropout_05_001` note even mentions its matched no-dropout run; retrieving that note is not proof that the *primary* run used no dropout. Open the linked JSON for exact values and config.

Chapter 11 - Our first search: keyword matching

The early `keyword_rank` function lowercases text, extracts alphanumeric words with a regular expression, and turns them into sets. It ranks notes by the count of shared words: `query_words & note_words`. For “validation loss rose,” a note containing those three exact words scores highly.

Figure 07 - Word overlap



The weakness is vocabulary mismatch. “Oscillating optimization” and “validation loss went up and down” may point to related behavior but share few exact terms. Keyword search is simple and inspectable; it cannot reliably infer synonyms or paraphrases. That gap motivates an embedding model.

> **You should now understand - checkpoint B:** (B1) Where do exact measurements live? (B2) Why is a note not a new experiment? (B3) When is exact filtering better than text similarity?

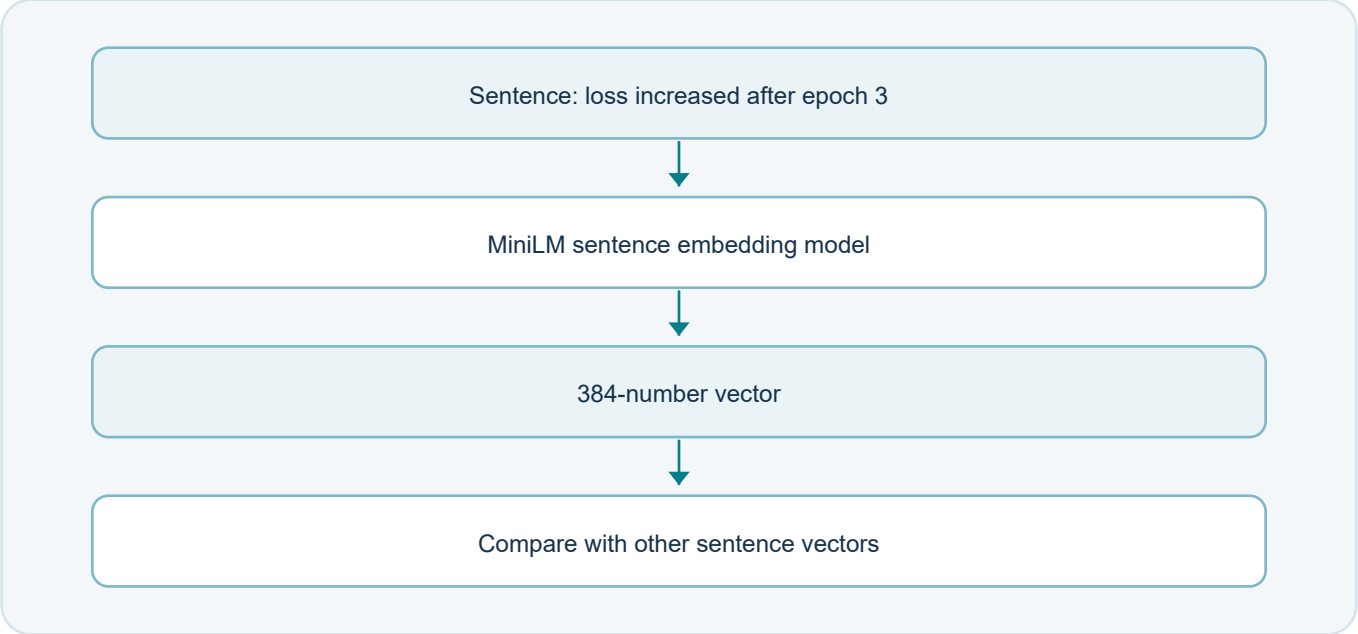
Part II - Find evidence: semantic retrieval and RAG

Chapter 12 - What is an embedding?

A computer compares numbers more easily than free-form sentences. An **embedding model** maps text into a fixed-length **vector**, an ordered list of numbers. In this project, `sentence-transformers/all-MiniLM-L6-v2` produces a **384-dimensional** vector. “Dimension” means one position in that list. Its cached model config records hidden size 384 and mean pooling into a sentence representation.

Illustrative vector: “validation loss increased” might become `[0.12, -0.08, 0.31, ...]`. Those first three numbers are **invented teaching values**; the real vector has 384 learned values. Dimension 1 is not labeled “validation,” dimension 2 is not “loss.” Meaning is distributed across the whole vector.

Figure 08 - Text becomes a numeric representation

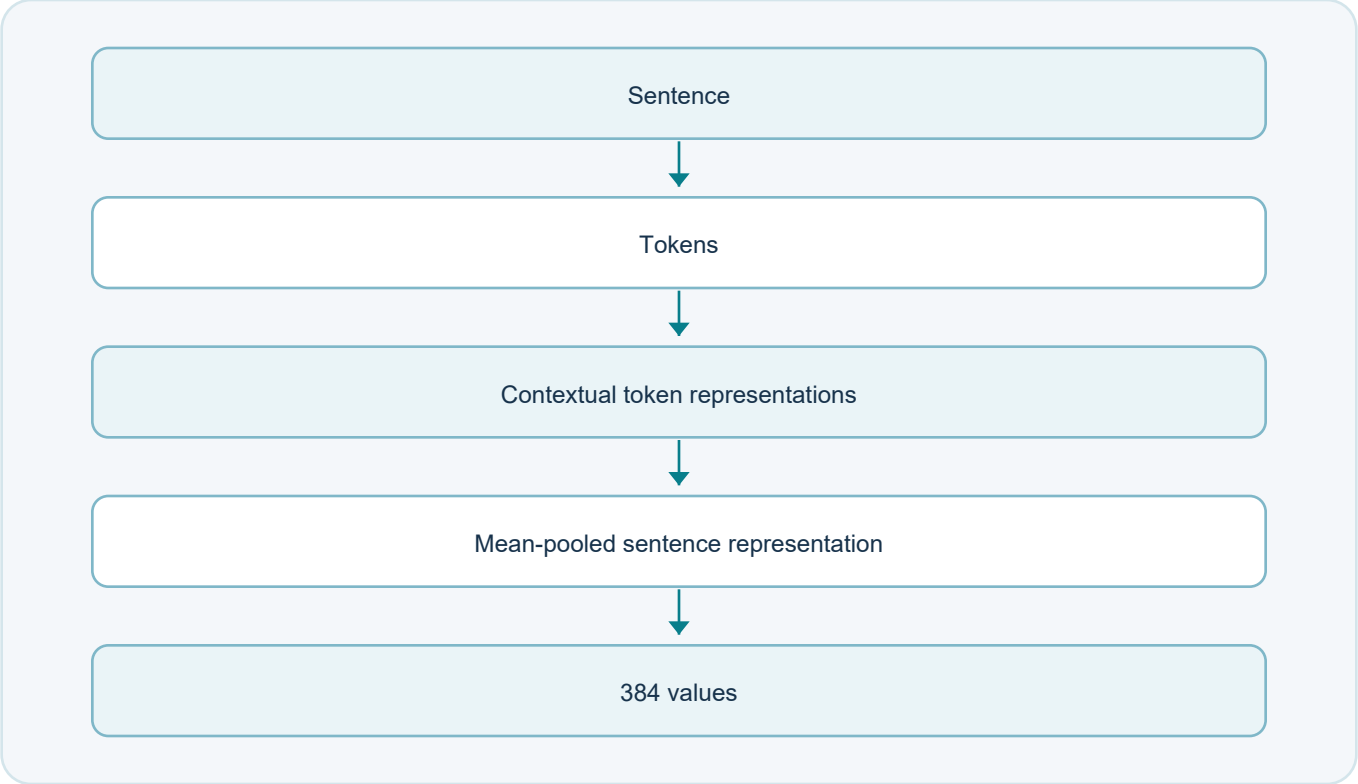


The model learned during earlier training to place some semantically related sentences in useful nearby regions. This is a ranking aid, not a truth detector. It can retrieve a plausible but wrong note.

Chapter 13 - Tokenization and the embedding model

A **token** is a piece of text processed by a language model, sometimes a whole word and sometimes a word part. Conceptually, the embedding path is: split sentence into tokens; a transformer lets each token representation depend on surrounding tokens; pool those representations into one sentence vector. The code calls `model.encode(...)` and receives vectors; it does not train MiniLM here.

Figure 09 - How a sentence reaches a vector

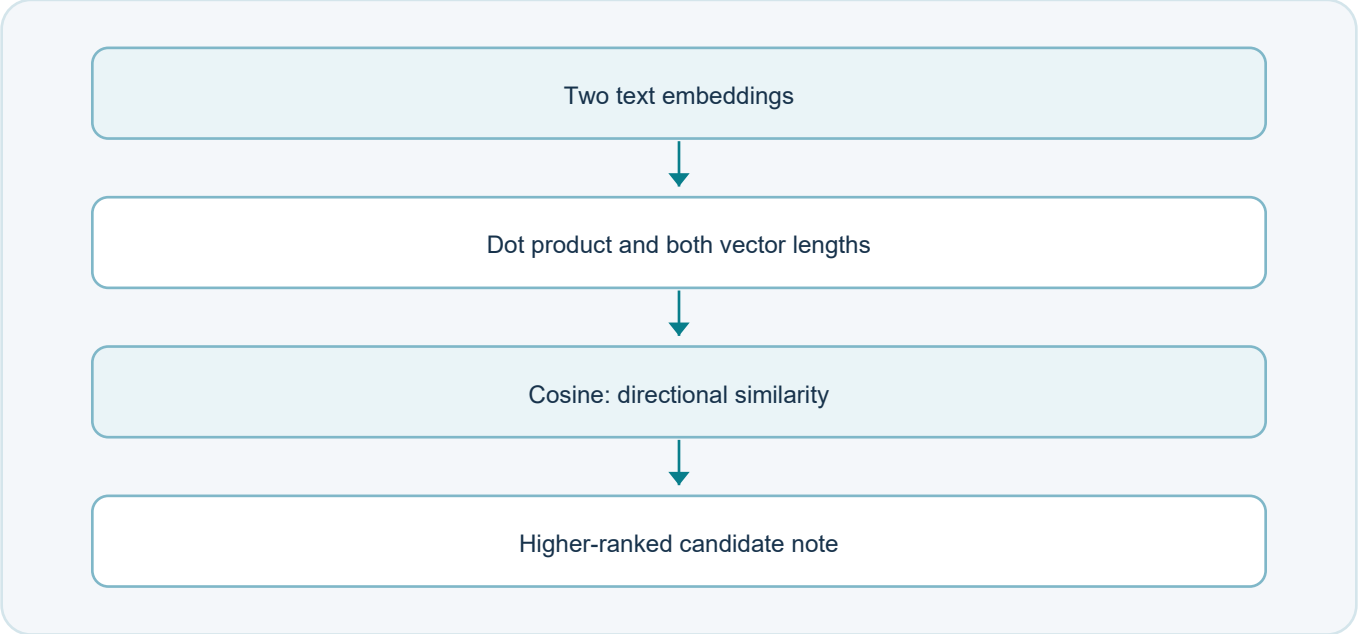


Keep two models separate: **MiniLM** turns text into vectors for retrieval. **qwen3:1.7b** is a generative LLM run through Ollama to interpret the question and choose actions. They have different inputs, outputs, and jobs.

Chapter 14 - Cosine similarity

Two arrows can point in the same direction even if one is longer. **Cosine similarity** measures how closely two vectors' directions align: `dot(a, b) / (length(a) * length(b))`. In a toy 2D example, `[1, 0]` and `[2, 0]` have cosine 1; `[1, 0]` and `[0, 1]` have cosine 0. The same calculation works with 384 numbers.

Figure 10 - Angle intuition

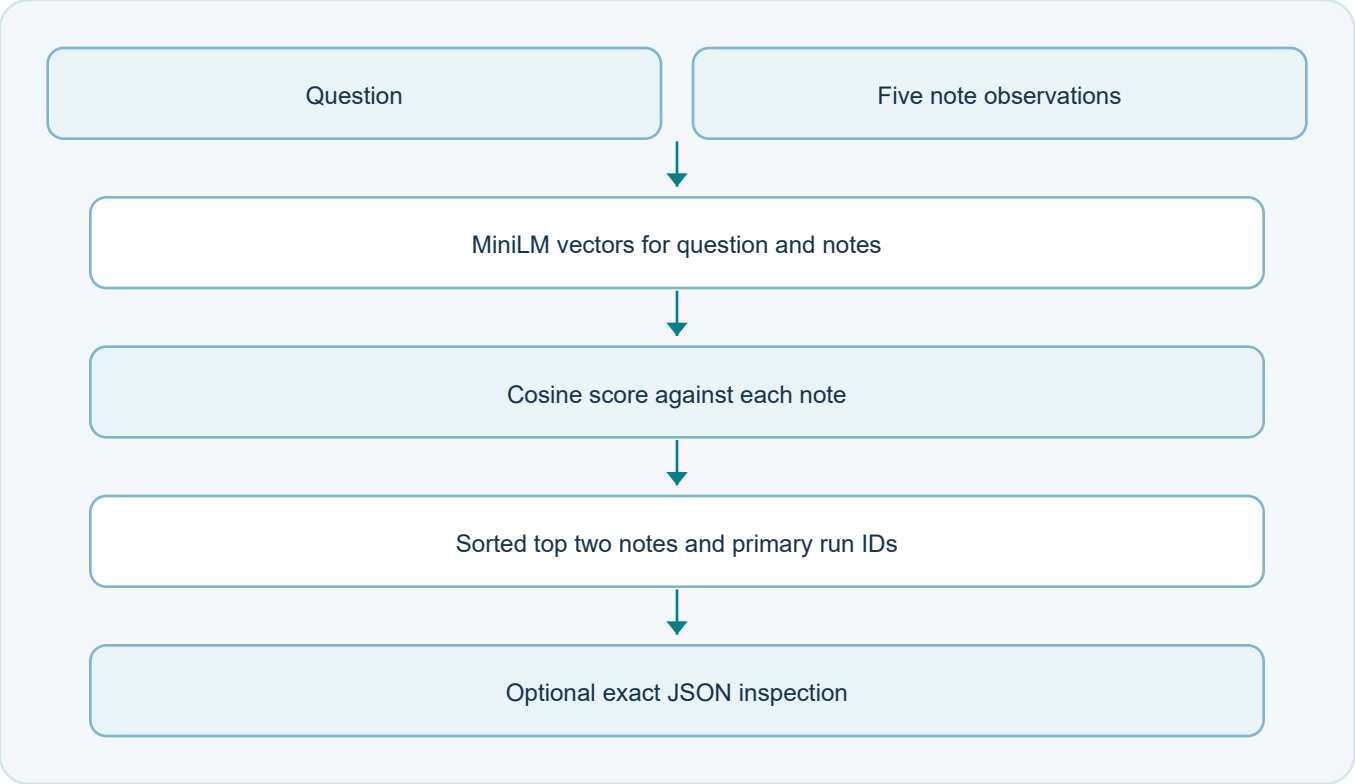


For the preserved query “unstable validation behavior,” `lr_01_seed2` scored `0.468` and `baseline_gap_001` scored `0.415`. These are **ranking scores for that query**. `0.468` does not mean a 46.8% probability that the note is correct, and a score alone cannot diagnose unstable training.

Chapter 15 - Semantic search

Semantic search uses meaning-oriented vectors instead of only matching words. `note_retrieval.py` loads each note's `Observation:` text, encodes the query and all observations, calculates cosine scores, sorts descending, and returns the top-k candidates. In the current `search_notes` tool, `top_k` defaults to two and the embedding model is loaded from `.model_cache/` with `local_files_only=True`.

Figure 11 - Semantic retrieval over notes



Five notes are cheap to encode and compare directly, so the project has no vector database or persistent vector index. At much larger scale, an index can avoid comparing every document each time. Search results are *candidates*. The app follows a candidate ID into its JSON record when it needs exact measurements.

Chapter 16 - Retrieval evaluation

An answer can look plausible even if search ranked the wrong note. `retrieval_eval_cases.json` stores five small, human-labeled questions and relevant primary note IDs. `evaluate_retrieval.py` compares keyword and semantic rankings at top two. **Recall@2** asks, “What fraction of the labeled relevant notes appeared in the first two?” If two notes are relevant and one appears, recall is 0.5. **MRR@2** gives 1 when the first relevant note is rank 1, 0.5 at rank 2, and 0 if absent.

The historical five-case report showed mean Recall@2 `0.90` for keywords and `1.00` for semantic retrieval, with MRR@2 `0.80` and `0.90`. This tiny, project-specific set is a teaching check, **not a general benchmark** or evidence that embeddings always win. Retrieval evaluation isolates a possible failure before blaming the LLM.

Chapter 17 - Structured filters and semantic search

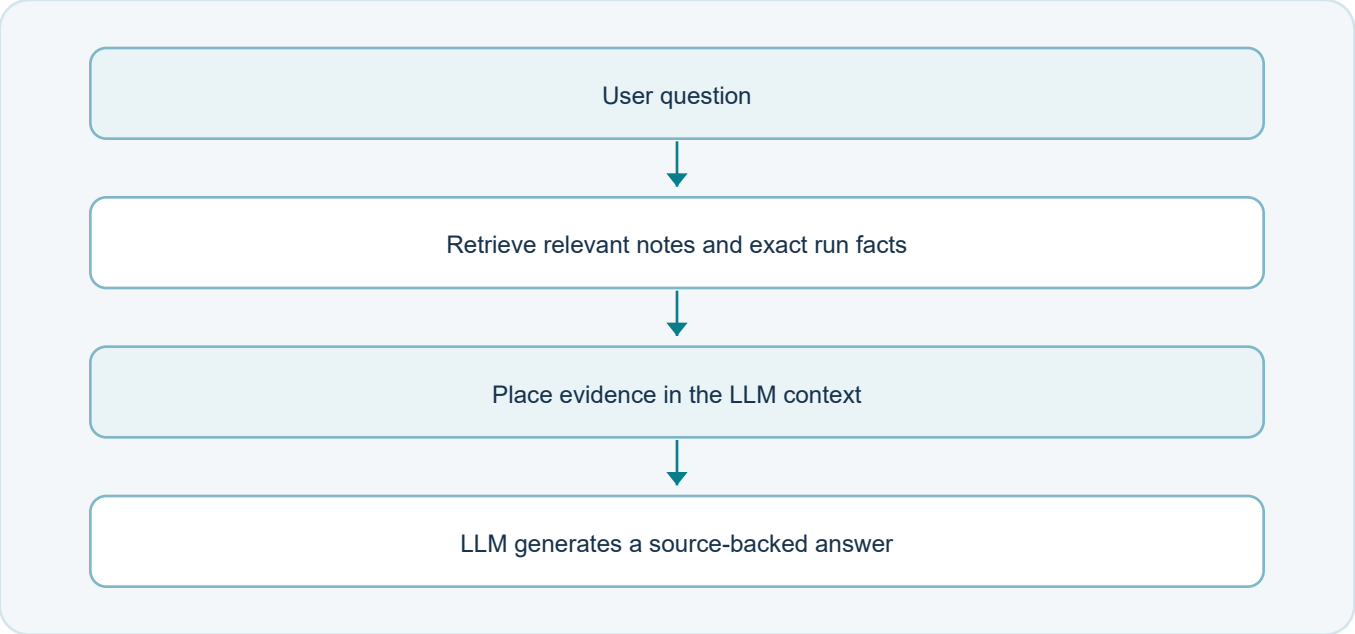
Text can mention a run other than the note's primary run. The dropout-0.5 note mentions a no-dropout comparison, so a vague query for “no dropout” can make that note look relevant. Historical `filter_notes` first checks the linked run JSON for exact constraints such as `dropout=0` and `train_size=1000`, then semantic ranking compares only eligible notes. The current agent's `search_notes` tool does **not** expose those filter arguments; the final app instead has a separate `search_runs` tool for exact config questions.

Principle: If the user states an exact recorded value, filter structured fields. If the user asks for a behavior described in prose, search meanings. If both are needed, filter candidates before interpreting text. Never infer the primary run's config from another run merely mentioned in a note.

Chapter 18 - What is RAG?

RAG means **Retrieval-Augmented Generation**. Retrieval finds outside information, augmentation puts that information into the model's working context, and generation produces an answer informed by it. A **context** is the text/messages available during one model call. **Grounding** means tying claims to that external evidence; it reduces, but does not eliminate, hallucination (plausible invented claims).

Figure 12 - A conventional RAG path



The project's earlier `build_context.py` / `prepare_answer_prompt.py` / `generate_answer.py` path followed that conventional generation pattern. **The current final app differs:** `search_notes` supplies retrieved context to Ollama so it can select more tools, but `render_answer` in Python writes the displayed final prose from successful tool results. Thus it is retrieval-augmented tool use and agentic retrieval; do not claim Ollama composes the final displayed answer. RAG does not require a vector database: keyword search, files, or direct vector comparison can supply retrieval.

Chapter 19 - Evidence packets

A historical **evidence packet** is like a case folder: the question on the cover, selected notes, actual run measurements, and source links inside. `build_context.py` created files under `packets/`. Retrieval identified candidate runs; Python added the underlying JSON rows so the LLM could see exact values. `prepare_answer_prompt.py` then built a constrained prompt, and `generate_answer.py` called Ollama with checks.

In the current agent, the packet is **in memory as tool results and message history**, not a new Markdown packet per question. `search_notes` returns note text and primary run IDs; `get_experiment` returns exact numbers; those results become tool messages. The distinction between a retrieved note and a measured JSON value remains essential.

> **You should now understand - checkpoint C:** (C1) Why do embeddings help when keywords fail? (C2) What does cosine 0.468 mean? (C3) How is semantic search different from RAG? (C4) Why can a note still mislead us?

Part III - Add a local model and tools

Chapter 20 - What is an LLM?

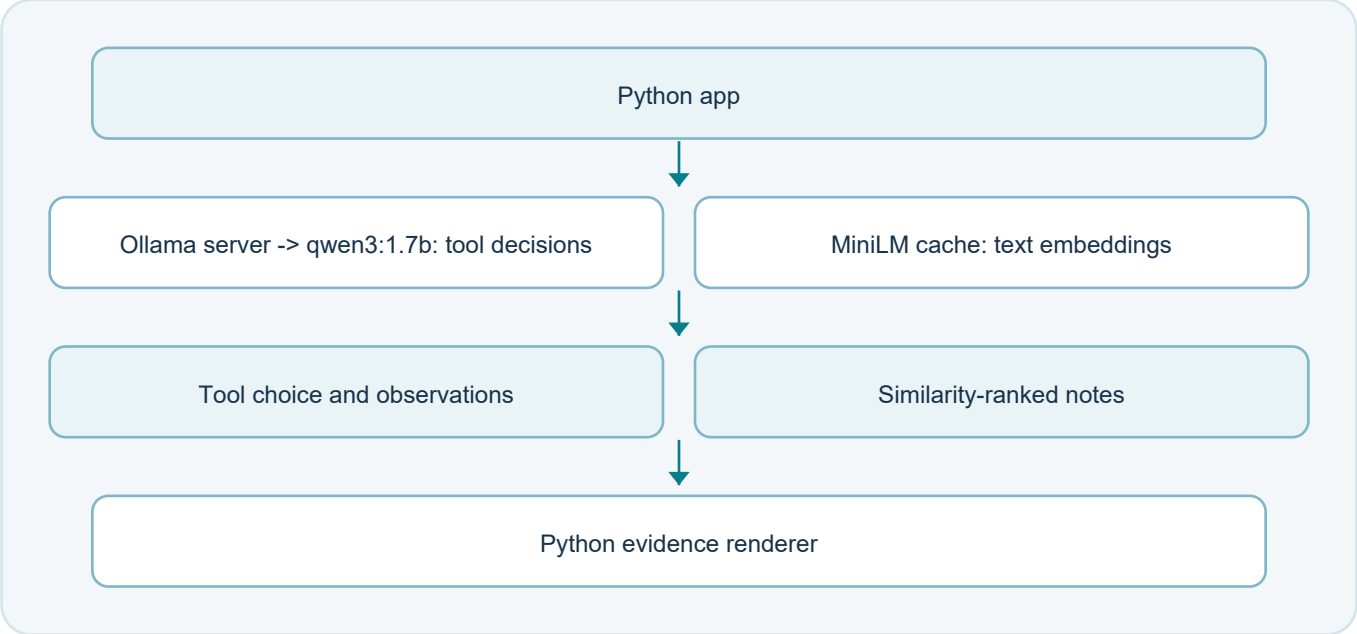
A **large language model (LLM)** reads a sequence of tokens in a limited context and predicts what text comes next, repeatedly. Instruction training lets it follow requests and produce useful language, but fluent text can be false. Here the local LLM handles a user's wording and chooses from tool descriptions. It does not have the saved run files inside its weights and does not execute Python by itself.

An LLM is useful for “this sounds like a behavior search” or “this question names two runs.” It is less reliable for exact file lookup, arithmetic, or checking whether a claimed cause follows from measurements. The application delegates those jobs to Python and the saved records.

Chapter 21 - What is Ollama?

Ollama is the local model runtime and HTTP server; **qwen3:1.7b** is the generative model it loads. Calling Ollama is like asking a local library desk to run that model; Ollama is not itself the model's learned content. `simple_agent.py` sends JSON to `http://127.0.0.1:11434/api/chat` using Python's standard library.

Figure 13 - Two local models, two jobs



The repository documents Ollama at `D:\Ollama`, model storage at `D:\Ollama\models`, and a `qwen3:1.7b` pull chosen for this machine's modest GPU memory. The final MVP does not require an OpenAI or Claude API key. The embedding model is cached separately in the project; Ollama does not create those note vectors.

Chapter 22 - Why not just ask Ollama everything?

An LLM can say “the learning rate caused the lower score” even when the records show only an observed difference. It may remember no run metrics at all or misstate a number while citing a correct file. The project therefore separates jobs: records hold evidence; retrieval finds candidates; Python reads and calculates; Ollama selects a tool; Python renders the final facts.

Tiny example: `0.759 - 0.7145 = 0.0445`, or 4.45 percentage points. The comparison function calculates target minus baseline as `-4.45 pp` for `lr_01_seed3` versus `lr_005_seed3`. A model should not invent a percent sign or claim that this one pair proves a mechanism.

> **Why we need this:** trustworthy answers need a path back to exact records and a way to see which operation produced each fact.

Chapter 23 - Tool calling

A **tool** is an action exposed to the LLM through a description and an argument schema. In this project it maps to a normal Python function. A **tool call** is the LLM's request to run one, not the execution itself. `TOOLS` in `simple_agent.py` declares four permitted actions:

Tool	Use it when...	It returns...
`get_experiment(run_id)`	One exact run ID is known	Config, final and epoch metrics, source, loss patterns
`compare_experiments(baseline_run_id, target_run_id)`	Two IDs need a measured comparison	Config changes, accuracy and loss differences
`search_notes(query, top_k)`	Behavior is described without a run ID	Top note observations, primary IDs, similarity scores
`search_runs(learning_rate, train_size, dropout)`	Exact numeric configuration is requested	JSON records matching stated fields

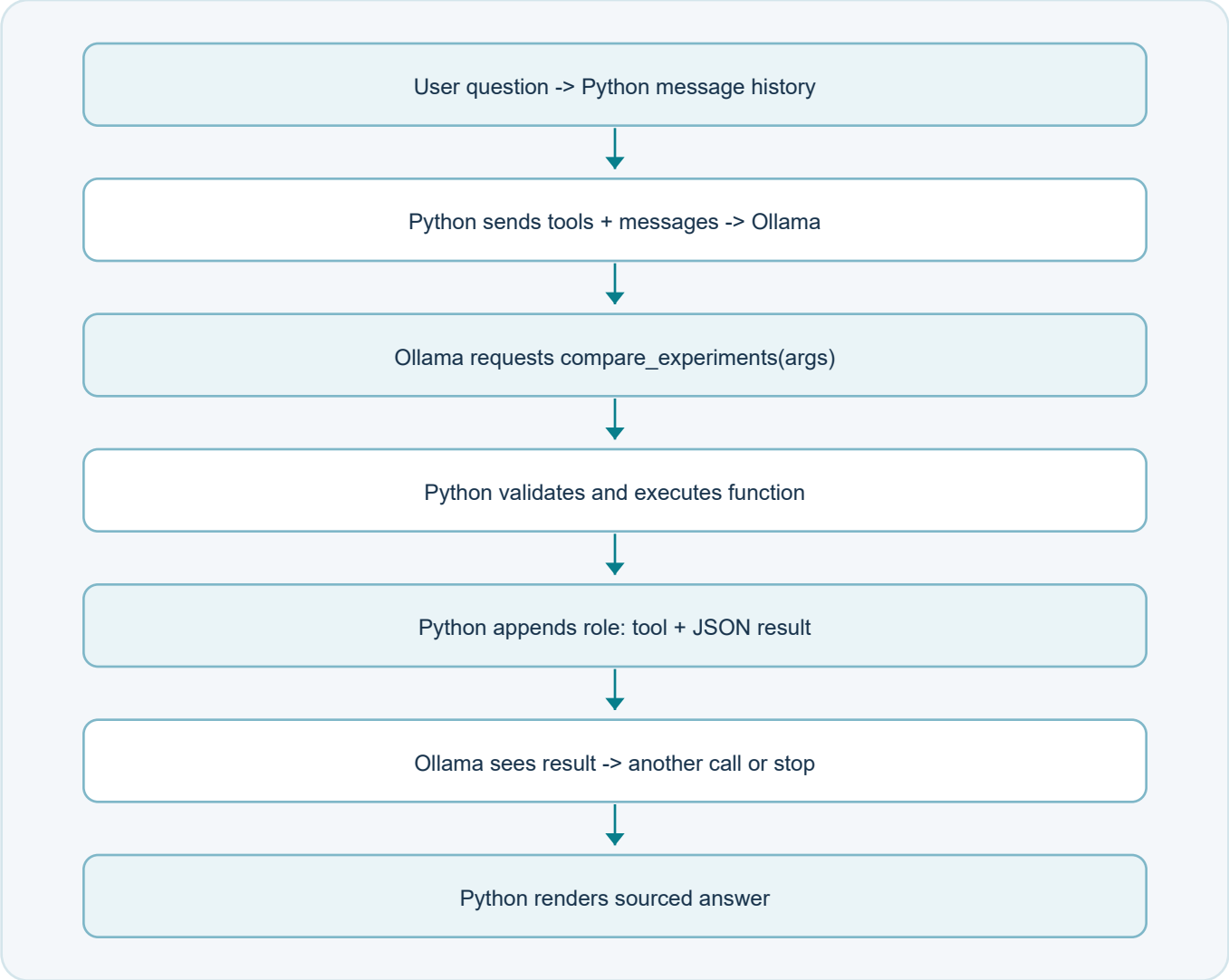
The **schema** says each tool's name, description, accepted argument types, and required fields. For example, `get_experiment` requires a string `run_id`; `search_notes` requires a string query and allows `top_k` from 1 to 5. The model sees schemas in the Ollama request and can choose a name and arguments. Python checks the name against `HANDLERS` and validates arguments before executing anything.

> **Remember this:** the model requests; Python performs. A tool is one possible action. An agent is the decision process around actions.

Chapter 24 - Tool-calling message flow

Suppose the user asks, “Compare lr_01_seed3 and lr_005_seed3.” Python recognizes two saved IDs and offers `compare_experiments` first. The model may return a structured `tool_calls` entry rather than ordinary prose. Python validates it, calls the function, serializes the result as JSON in a `role: "tool"` message, and asks Ollama again. The returned numbers are the model's **observation** for its next decision.

Figure 14 - Request, execution, observation



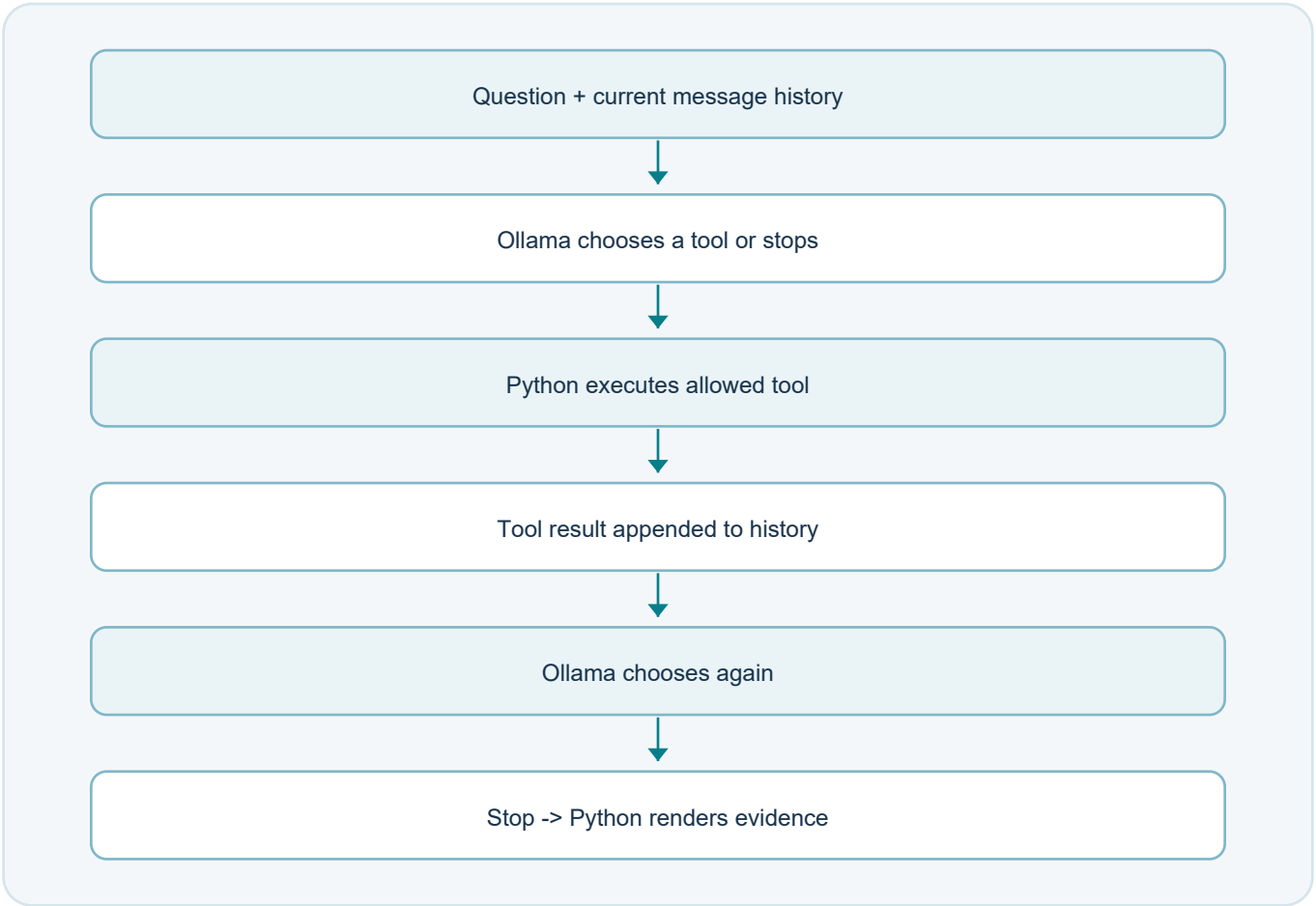
The `role: "assistant"` message containing the requested call is also preserved before the tool result. This keeps the next model turn coherent: it can see what it asked for and what came back. An API is simply the agreed format by which Python communicates with Ollama; here it is local HTTP plus JSON.

Chapter 25 - What is an agent?

In this project an **agent** is a small control loop with a model, available tools, message history (short-lived **state**), and a stopping condition. The model proposes the next action from what it currently knows; Python runs that action and returns an observation; the model can then choose again. Calling one function once is tool use, but the choose-execute-observe-repeat cycle is what makes this application agentic.

The agent is **single-agent**: one model decision stream, not a team of models. Its scope is bounded by four tools and at most five calls. Some first-tool choices are constrained by Python so the small model starts with the right class of evidence.

Figure 15 - The agent loop



Chapter 26 - The agent loop in our project

`answer_question` in `simple_agent.py` first extracts any exact saved run IDs from the question and chooses an initial tool menu. Two IDs offer comparison; one offers inspection; a configuration question offers structured search; a behavior question without IDs offers note search. After a first search, other follow-up tools may be available. This routing is **application logic**, not the model discovering every option independently.

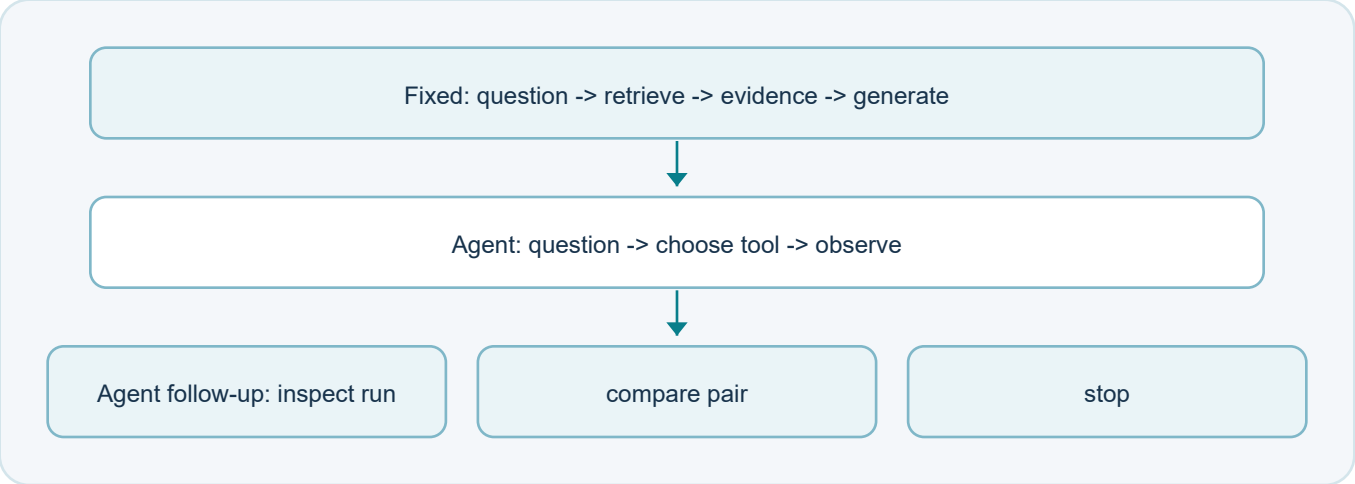
The loop calls `ask_ollama(messages, available_tools)`. If the response contains `tool_calls`, Python validates each request with `execute_tool`, appends the assistant request and `role: "tool"` result, and loops. If no call arrives after evidence was collected, it stops. `MAX_TOOL_CALLS = 5` prevents an endless sequence; exceeding it raises an error rather than silently producing unsupported prose. Tool errors are sent back as error results; only successful results feed the final renderer.

> **What can go wrong?** The LLM can choose weak search wording, wrong arguments, or an unhelpful follow-up. A bounded loop and a visible trace make those mistakes inspectable. The current app does not solve all arbitrary user questions.

Chapter 27 - Fixed RAG pipeline versus agentic retrieval

A fixed RAG script follows the same steps for every question: retrieve, place evidence in context, generate. The current agent has different possible paths based on the question and tool observations. It can read an exact run without semantic search, compare a named pair, filter config fields, or search notes and then inspect returned IDs.

Figure 16 - Fixed path and choice-based path

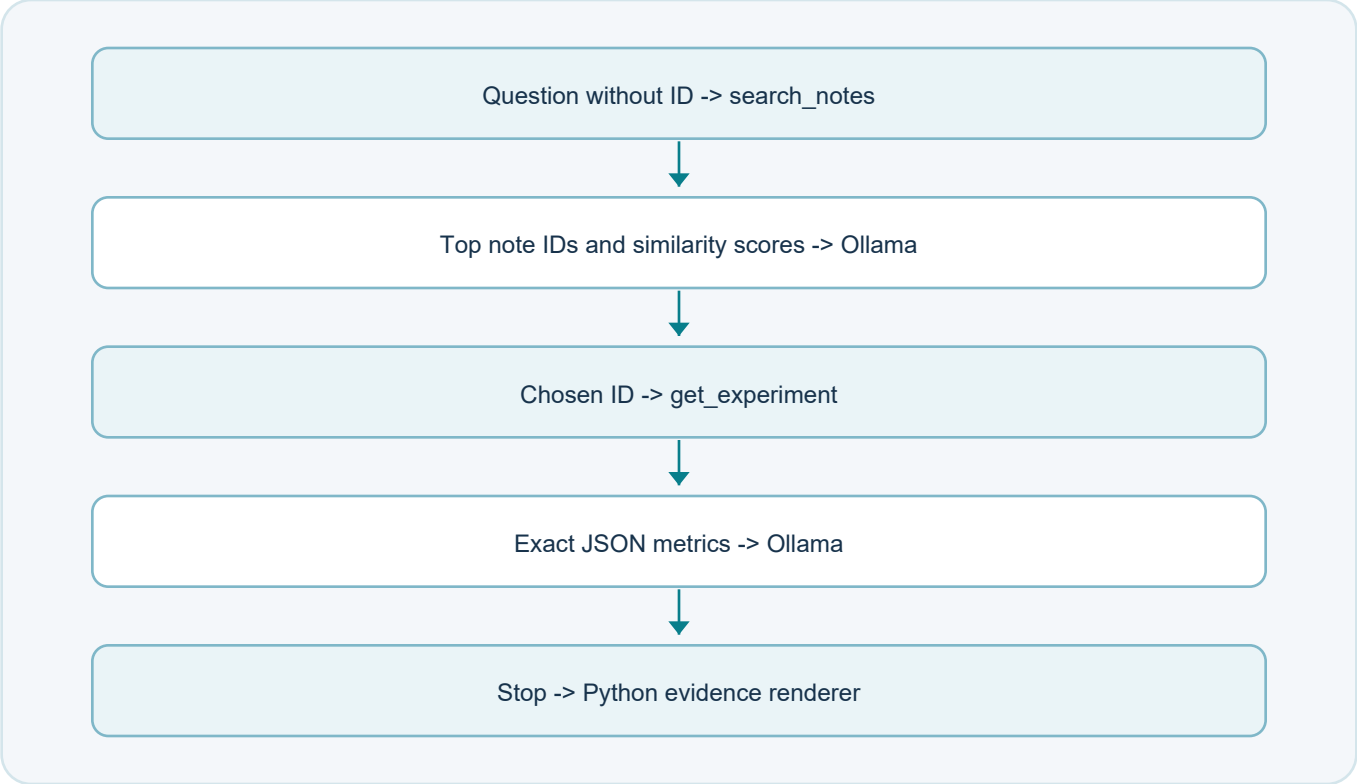


The agentic property is **conditional choice and iteration**, not merely the presence of an embedding model. In this MVP the first choice is partly narrowed by Python rules, and Python sometimes enforces a follow-up. Those rules make the system easier to teach and more reliable with the local model.

Chapter 28 - RAG as an agent tool

`search_notes(query)` packages semantic retrieval as one callable tool. With “Have we seen unstable validation behavior before?”, no run ID is supplied, so search can discover relevant note IDs. Then `get_experiment` can inspect exact JSON. For “What is the final validation accuracy of lr_01_seed3?”, the run ID is already present, so `get_experiment` is the direct path. Retrieval is available **when needed**, not forced into every question.

Figure 17 - Semantic retrieval inside tool use



This combines a RAG *retrieval component* with an agent loop. The final displayed prose is deterministic Python output, so this particular final path is not “LLM writes the final answer from retrieved context.” Chapter 18's conventional RAG description remains useful because the earlier generation path exists in the repository and explains why retrieved context matters.

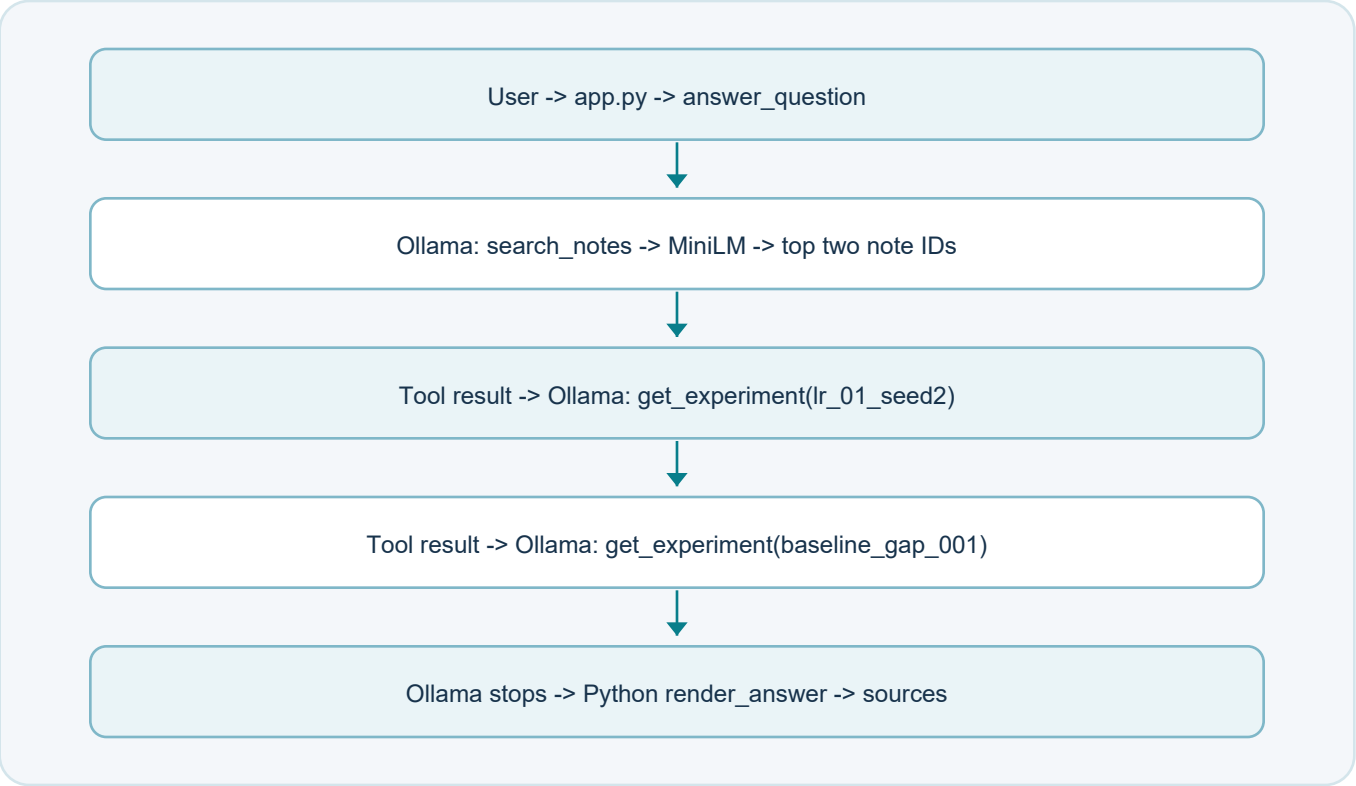
> **You should now understand - checkpoint D:** (D1) Who executes a requested tool? (D2) Why can `get_experiment` skip semantic search? (D3) What makes the loop agentic? (D4) Why bound tool calls?

Chapter 29 - Complete end-to-end trace

This chapter follows the preserved real trace in `docs/TRACE_EXAMPLE.md`. Tool decisions can vary on another run. The original question was **“Have we seen unstable validation behavior before?”** There is no exact run ID in the wording.

1. `app.py` receives the question and calls `answer_question(question, trace=True)`.
2. Python creates system and user messages; the first menu offers `search_notes` because the question names no run and asks about behavior.
3. `ask_ollama` sends those messages, tool schema, model `qwen3:1.7b`, and options to the local Ollama `/api/chat` endpoint.
4. Ollama requests `search_notes({"query": "unstable validation behavior"})`.
5. Python validates the name and argument. `search_notes` loads all five `Observation:` texts.
6. Cached MiniLM converts the query and observations into 384-value embeddings; `semantic_rank` computes cosine scores.
7. The top two are `lr_01_seed2` at `0.468` and `baseline_gap_001` at `0.415`. These scores rank descriptions, not probabilities.
8. Python returns note text, primary IDs, scores, and paths in a `role: "tool"` message.
9. Ollama asks for `get_experiment({"run_id": "lr_01_seed2"})`. Python reads its JSON and reports `0.7136 -> 0.8142 -> 0.6793` over epochs 3 to 5, plus final validation accuracy `76.35%`.
10. Ollama asks for `get_experiment({"run_id": "baseline_gap_001"})`. Python reads `0.8696 -> 1.4186 -> 0.6663` over epochs 1 to 3, with final validation accuracy `71.25%`.
11. Ollama stops after three tool calls. Python's `render_answer` formats the inspected observations and loss patterns and lists the four note/JSON sources. It does **not** publish Ollama's free-text explanation.

Figure 18 - The complete question journey



The final answer is a **yes, with examples**, not a claim about the cause of instability. Appendix A contains the detailed compact trace and the source paths. Note that the preserved trace is historical: the present `.venv` could run direct JSON lookup during this PDF build, but its semantic dependency lacked `typing_extensions`, so a fresh live semantic replay could not be verified without repairing that environment.

Chapter 30 - Complete architecture

Read this architecture from top to bottom. Data generation happens in `phase0.py`; question answering happens in `app.py` and `simple_agent.py`. `experiment_data.py` is the exact-record worker, while `note_retrieval.py` is the meaning-based search worker. Both return evidence to the agent's message history. The final renderer uses those results and adds source paths.

Figure 19 - Local system architecture

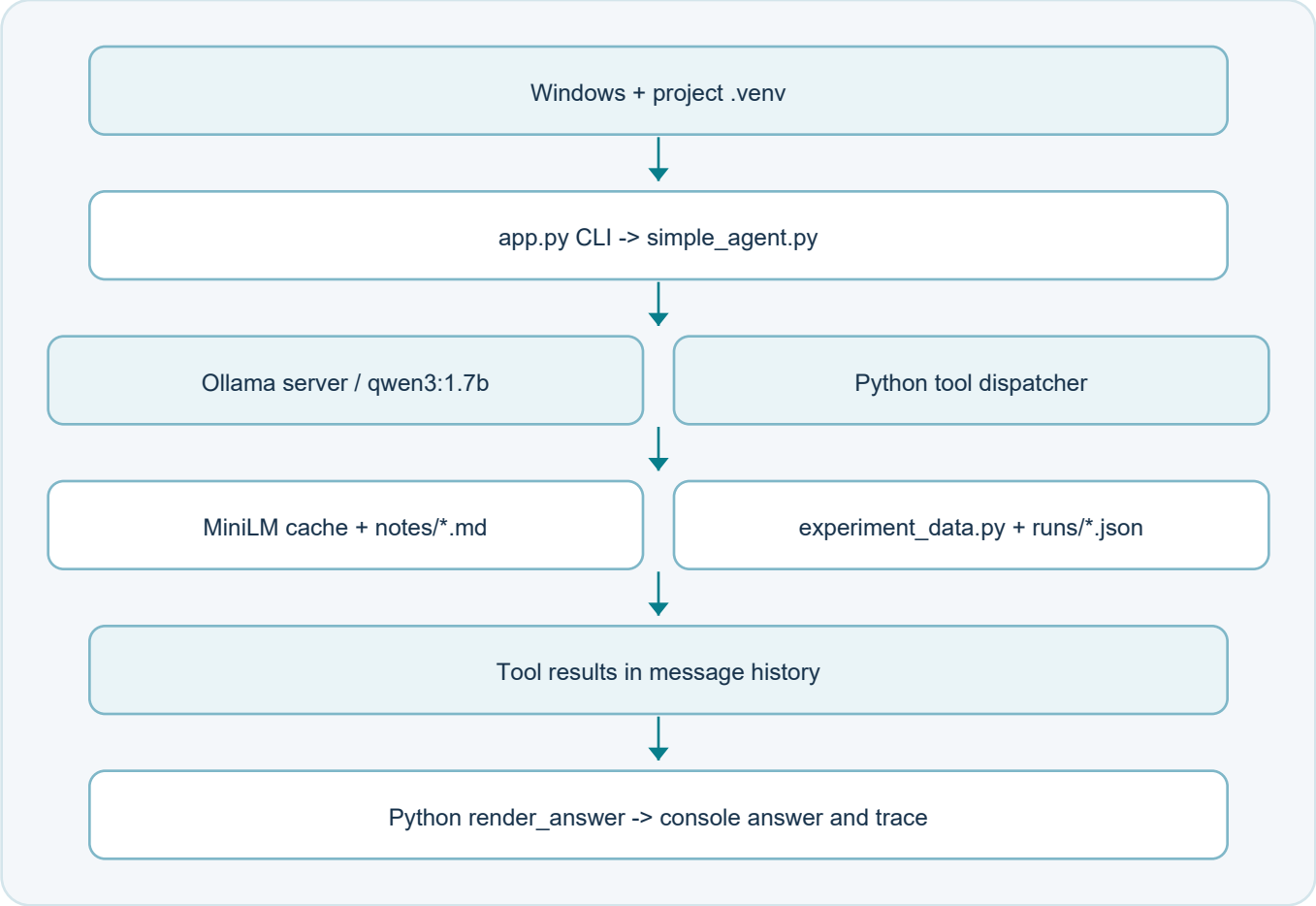
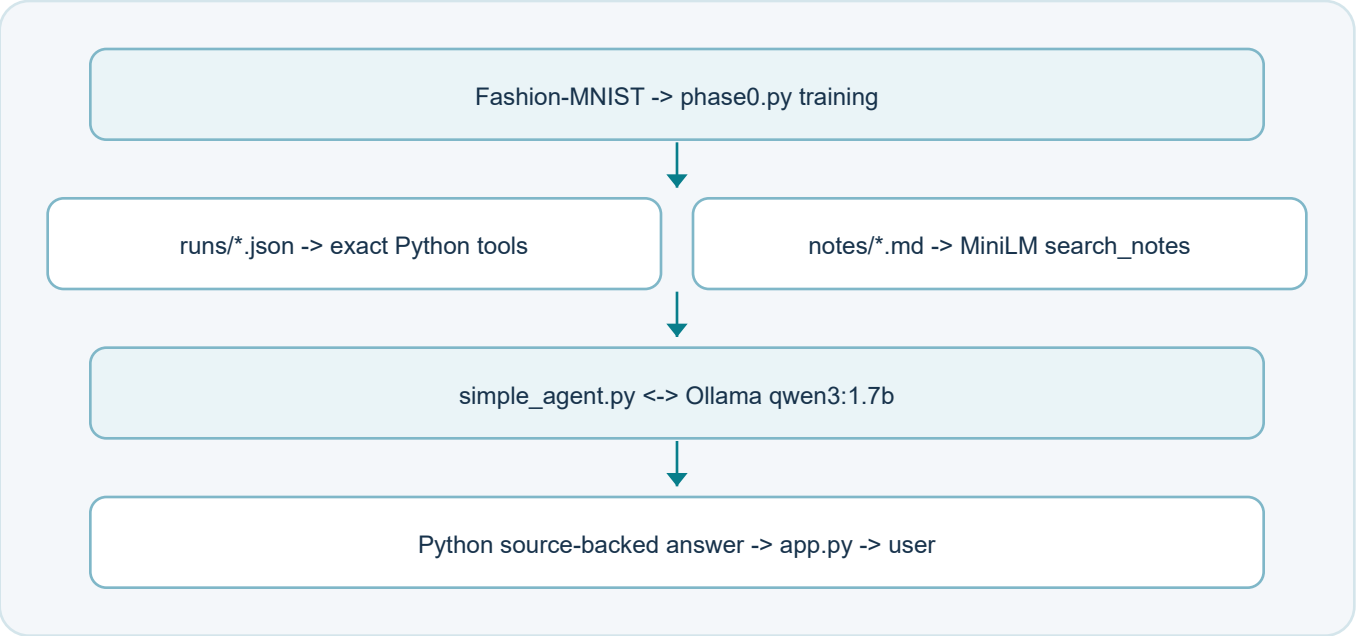


Figure 20 - Full project, including evidence creation

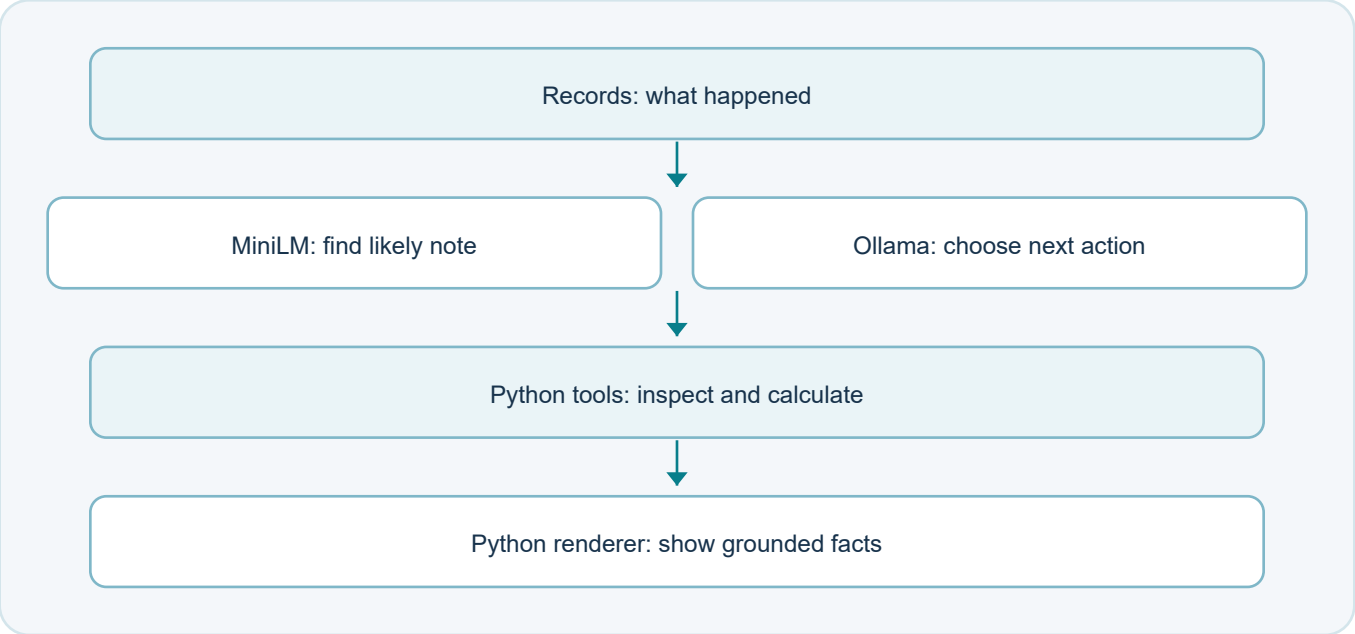


The arrows into Ollama represent messages, not direct file access. The arrow from `runs/*.json` to Python represents exact reading and calculation. The MiniLM branch ranks note wording; it does not replace JSON evidence.

Chapter 31 - Responsibility separation

Component	Its reliable job	Do not confuse it with...
`runs/*.json`	Saved configurations and measurements	A theory of why a metric changed
`notes/*.md`	Human-readable observation clues	Independent measurements
MiniLM embedding model	Text-to-vector ranking	A generative answer model
Ollama + qwen3	Understand question; request tools; stop	Python execution or exact arithmetic
Python tool functions	File access, validation, filtering, arithmetic	Natural-language understanding
Python `render_answer`	Publish measured results with sources	Free-text LLM generation

Figure 21 - Responsibility handoff



Splitting jobs makes errors traceable. If the wrong note appears, inspect retrieval. If a metric is wrong, inspect JSON or Python calculation. If a strange tool is chosen, inspect the model request and the offered schema.

Chapter 32 - APP POLICY and APP FILTER

The trace labels decisions made by **application code** rather than by Ollama. If Ollama stops immediately after `search_notes` without inspecting a run, `APP POLICY` opens the highest-ranked returned run with `get_experiment` so the final answer has exact metrics. If a structured question states a numeric filter and Ollama adds an unsupported one, `APP FILTER` replaces the arguments with the exact filters Python extracted from the user's wording.

For example, the question “Which experiments used learning rate 0.1?” should filter by `learning_rate=0.1`. A historical model call also added `dropout=0.1`; Python removed that invented constraint. This protection handles simple stated numeric patterns, not every possible form of language. The trace tells the user which calls came from `AGENT` and which corrections came from the app.

Chapter 33 - Local execution

The project's `.venv` is a **virtual environment**: a local collection of Python packages separate from the system Python. The command `./.venv/Scripts/python.exe app.py` (PowerShell form: `.\.venv\Scripts\python.exe app.py`) starts the CLI. Ollama must be serving on localhost and have `qwen3:1.7b`; the MiniLM cache must exist for semantic search.

Starting `app.py` does not retrain Fashion-MNIST. It prints a prompt, waits for a question, routes it through the agent, prints the trace by default, then prints the answer. Type `exit` to leave. A `--question` option asks one question and exits; `--no-trace` hides the verbose tool details. Appendix C gives commands checked against this checkout and identifies the local dependency issue found during PDF production.

Chapter 34 - Project file walkthrough

Read the current working path in this order:

File or folder	Purpose	Look for first	Leave for later
`app.py`	CLI entry point	`main`, question loop	`argparse` details
`simple_agent.py`	Schemas, Ollama call, loop, renderer	`TOOLS`, `execute_tool`, `answer_question`, `render_answer`	Every prompt sentence
`experiment_data.py`	Exact JSON worker	`load_run`, `get_experiment`, `compare_experiments`, `search_runs`	Edge-case validation
`note_retrieval.py`	Keyword and semantic ranking	`load_notes`, `encode_queries_and_notes`, `semantic_rank`	Revision hash
`runs/`	11 saved run records	One config and its epoch rows	Reading all 11 at once
`notes/`	5 observation summaries	Primary run ID and `Observation:`	All prose at once
`requirements.txt`	Semantic-search package pin	`sentence-transformers==6.1.0`	Transitive package details
`docs/`	Existing architecture, trace, and this guide	`TRACE_EXAMPLE.md`	Historic notes
`phase0.py`	Training laboratory	split, model, loop, JSON save	Re-running old experiments

Historical comparison, packet, answer, hypothesis, and evaluation scripts show how the system evolved. They are **not** startup requirements for `app.py`.

Chapter 35 - Important code walkthrough

These are short excerpts or faithful contractions of the inspected files. For each, read the code, then the three explanations. Omitted lines are marked `...`; do not paste a contraction as a standalone program.

35.1 CLI entry point - `app.py`

```
question = input("\n> ").strip()
answer = answer_question(question, trace=not args.no_trace)
print(answer)
```

What it does: reads one question, calls the agent, prints its returned answer. **Why it exists:** gives a simple local interface. **How it connects:** `app.py` owns interaction, while `simple_agent.py` owns the reasoning-and-tool cycle.

35.2 Tool schema - `simple_agent.py` (abbreviated)

```
{"type": "function", "function": {
    "name": "get_experiment",
    "parameters": {"type": "object", "properties": {
        "run_id": {"type": "string"}}, "required": ["run_id"]}
}}
```

What it does: tells Ollama the permitted tool name and the shape of its arguments. **Why it exists:** the model needs an explicit contract to request a valid call. **How it connects:** Python later checks the same name and arguments before opening a run.

35.3 Allowlist dispatcher - `simple_agent.py`

```
if name not in HANDLERS:
    raise ValueError(f"Tool is not allowed: {name}")
function, required, optional = HANDLERS[name]
return function(**arguments)
```

What it does: maps an approved name to a Python callable, after further argument checks in the omitted lines. **Why it exists:** a model's request must not become arbitrary code execution. **How it connects:** this is the bridge from model-selected action to exact Python work.

35.4 Local Ollama request - `simple_agent.py` (selected lines)

```
body = {"model": MODEL, "messages": messages,
        "tools": available_tools, "stream": False, "think": False,
        "options": {"temperature": 0, "num_ctx": 4096, "num_predict": 500}}
request = Request("http://127.0.0.1:11434/api/chat", ...)
```

What it does: sends current state and tool choices to `qwen3:1.7b` through Ollama. **Why it exists:** the LLM needs both conversation and tool descriptions for a decision. **How it connects:** the returned `message` may contain `tool_calls`; the HTTP request itself executes no experiment tool.

35.5 Agent loop - `simple_agent.py` (faithful pseudocode)

```
for _ in range(MAX_TOOL_CALLS + 1):
    message = ask_ollama(messages, available_tools)
    calls = message.get("tool_calls") or []
    if not calls:
        return render_answer(tool_results)
    for call in calls:
        result = execute_tool(name, arguments)
        messages.append({"role": "tool", "content": json.dumps(result)})
```

What it does: asks, executes requested calls, records observations, and eventually stops. **Why it exists:** later decisions can use earlier results. **How it connects:** the real code also preserves the assistant call, enforces a limit, handles errors, and applies `APP POLICY` / `APP FILTER`.

35.6 Exact run retrieval - `experiment_data.py`

```
record = load_run(run_id)
final = record["metrics"][-1]
final_accuracy = round(100 * final["val_accuracy"], 2)
```

What it does: reads the named JSON and converts a fraction to a percentage. **Why it exists:** the answer needs the saved number, not a model guess. **How it connects:** `get_experiment` returns that value, epoch rows, config, and source to the agent.

35.7 Comparison - `experiment_data.py`

```
baseline, target = load_run(baseline_run_id), load_run(target_run_id)
change_pp = round(100 * (right["val_accuracy"] - left["val_accuracy"]), 2)
```

What it does: computes target minus baseline in percentage points after loading both records. **Why it exists:** direction and units matter. **How it connects:** `compare_experiments` also checks equal final epochs and reports changed recorded settings.

35.8 Embedding generation - `note_retrieval.py` (selected lines)

```
model = SentenceTransformer(MODEL_ID, cache_folder=str(PROJECT_DIR / ".model_cache"),
                            local_files_only=local_only, revision=MODEL_REVISION)
return model.encode(queries + [observation for _, observation in notes])
```

What it does: converts queries and note observations into vectors. **Why it exists:** semantic ranking needs numeric representations. **How it connects:** `search_notes` calls this with `local_only=True`, then pairs each vector with its note.

35.9 Cosine ranking - `note_retrieval.py`

```
score = float(np.dot(first, second) /
              (np.linalg.norm(first) * np.linalg.norm(second)))
```

What it does: measures vector-direction similarity. **Why it exists:** ranks likely relevant notes for a meaning-based query. **How it connects:** `semantic_rank` sorts these scores and `search_notes` returns the top candidates.

35.10 Final evidence renderer - `simple_agent.py` (selected line)

```
return render_answer(tool_results)
```

What it does: formats successful tool results as the displayed answer. **Why it exists:** the small model previously misstated numbers and causes in free prose. **How it connects:** the final CLI answer comes from measured evidence and path citations, while Ollama's role ends at tool choice and stopping.

Chapter 36 - What we learned from failures

Keyword matching missed related wording, motivating MiniLM retrieval. Semantic search itself can rank a note that talks about another run, motivating exact config checks. A small LLM generated unsupported causal language and sometimes wrong or missing numbers even while citing sources, motivating deterministic Python calculations and final rendering. It also added a filter the user never requested, motivating `APP FILTER`; it sometimes stopped after note search, motivating `APP POLICY`.

These are design lessons rather than proof that every question is now safe. The application is intentionally a small, inspectable learning system. Its trace makes a bad tool choice visible, and its source paths let a reader compare the answer with saved records.

Chapter 37 - RAG versus Agentic AI

Question	RAG	Agentic AI in this app
Main idea	Find external information to support a response	Choose permitted actions based on question and observations
Core step	Retrieval places evidence in context	Model requests a tool; Python executes; model observes
Project example	MiniLM ranks experiment notes	Ollama chooses `search_notes`, then perhaps `get_experiment`
Final displayed prose	Conventional RAG: often generated by LLM	Here: rendered by Python from successful tool results

These ideas overlap. The current system makes semantic retrieval one agent tool. **Semantic search** alone only finds candidates; it does not answer with verified run metrics. **RAG** is retrieval plus answer production using external context. **Agentic AI** adds action choice and iteration. The earlier `generate_answer.py` demonstrates LLM generation after retrieval, while today's CLI demonstrates agent-selected retrieval with a deterministic published answer.

Chapter 38 - What this project is not yet

There are five notes and eleven saved run records, not a large knowledge base. There is no vector database, long-term conversational memory, LangGraph graph, multi-agent team, or autonomous training. The test partition remains outside the training script's current comparison path. Similarity scores can be misleading, notes can be imprecise, and a five-call limit can stop a difficult query. The app can describe observed settings and metrics; it cannot prove that one hyperparameter caused an outcome.

These boundaries are useful. A small local system exposes each concept without hiding it behind an orchestration framework. Reliability measures cover demonstrated question types but do not constitute a general scientific reasoning guarantee.

Chapter 39 - Where LangGraph would fit later

The manual loop already has state (messages and results), actions (tool calls), transitions (after a result, ask again), and termination (stop or call limit). A future graph framework such as LangGraph could represent a more elaborate flow as named nodes and edges. This is a possible location, **not a component of the current application**. Learning the explicit loop first shows what a framework would organize.

Chapter 40 - Where a vector database would fit later

Today `search_notes` encodes five notes and computes five similarities directly. A much larger note collection could use a persistent vector index to store embeddings and locate near neighbors faster than scanning every item. **Approximate nearest-neighbor** search is a speed trade-off: it may return likely close vectors without checking every single one. It would sit between MiniLM embeddings and candidate note IDs. It would not replace run JSON as the source of exact measurements.

Chapter 41 - Where memory would fit

The current CLI treats each question as a new agent call. **Conversation memory** would remember prior turns so “Compare that run with seed3” could resolve “that run.” Saved JSON and notes are **experiment knowledge**, not conversation memory. Adding memory would require rules for what to retain and how to avoid carrying an incorrect reference into a later answer. It is not implemented here.

Chapter 42 - Where multi-agent would fit

Multi-agent means multiple decision-making model roles exchanging tasks, perhaps a researcher, an experiment analyst, and a reviewer. This project has one tool-using model loop plus Python functions. More agents would add coordination and more opportunities for mistakes; they are not automatically better for eleven runs. The single-agent design is sufficient to learn the action-observation cycle.

Chapter 43 - Glossary

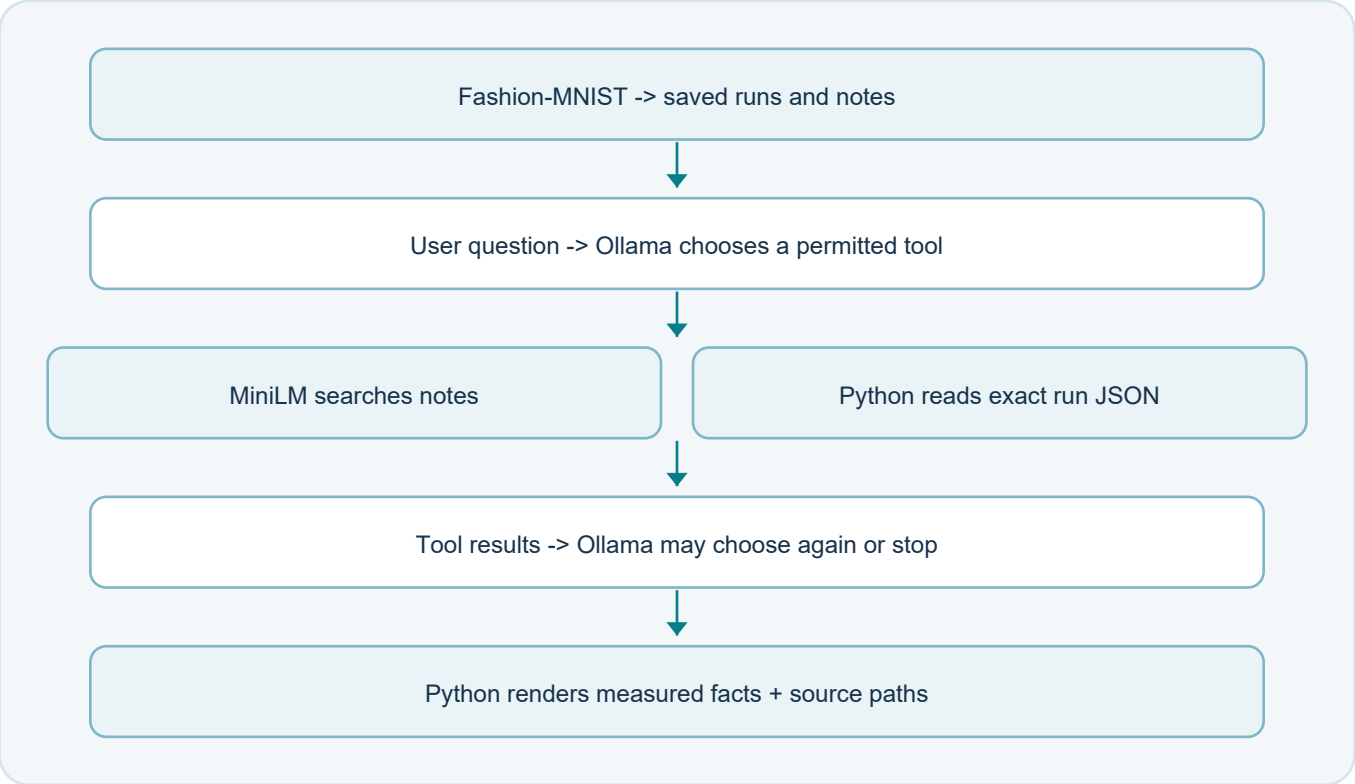
Term	Plain-language meaning
Dataset	Collection of examples for a task
Sample	One example, such as one clothing image
Label	Correct category attached to a sample
Training set	Examples used to update model weights
Validation set	Separate examples used to monitor generalization
Test set	Held-back examples for a later final check
Batch	Group processed for one model update
Epoch	One pass through the selected training set
Neuron	Learned calculation that combines inputs, weights, and bias
Weight	Learned strength of an input connection
Bias	Learned additive adjustment to a neuron
Hidden layer	Intermediate layer between input and output
ReLU	Function that replaces negative values with zero
Logit	Unnormalized output score for a class
Loss	Number measuring prediction error under a chosen rule
Optimizer	Procedure that updates weights using gradients
Learning rate	Size control for optimizer updates
Seed	Starting value for a repeatable pseudo-random sequence
Run / experiment	One training execution and its saved record
Hyperparameter	Setting chosen before a training run
Structured data	Values in fixed named fields, such as JSON config
Unstructured data	Free-form text, such as a note observation
Token	Text piece processed by a language model
Embedding	Numeric vector representing text for comparison
Vector / dimension	Ordered number list / one position in that list

Term	Plain-language meaning
Sentence transformer	Model that turns a whole sentence into a vector
Cosine similarity	Directional similarity score between vectors
Keyword search	Retrieval by overlapping literal words
Semantic search	Retrieval using meaning-oriented representations
Retrieval	Finding candidate external information
Top-k	First k items in a ranked result list
Recall@K	Fraction of labeled relevant items among first K
MRR	Average reciprocal rank of first relevant result
Context	Text/messages a model can currently use
LLM	Model that processes tokens and generates language or tool requests
Hallucination	Plausible output unsupported by the evidence
Grounding	Connecting claims to inspected external sources
RAG	Retrieval-augmented generation: outside evidence informs an answer
Ollama	Local server/runtime that runs the generative model
API	Agreed interface for programs to exchange requests and results
JSON	Text format for named fields, arrays, and values
Tool / schema	Allowed action / description of its arguments
Tool call / result	Model request for action / Python's returned observation
Agent / agent loop	Decision process with tools / choose-execute-observe cycle
State	Current messages, results, and other working information
Source / citation	Path showing which saved file supports a claim
Vector database	Storage/index system for large embedding collections
LangGraph	Optional future graph orchestration framework
Memory	Stored prior conversational context for later questions
Multi-agent	Several model-driven roles coordinating tasks

Chapter 44 - Final mental model

- **Experiment store:** what happened.
- **Notes:** clues written about it.
- **MiniLM:** turn note meaning into vectors.
- **Retrieval:** find likely clues.
- **Ollama/qwen3:** understand the question and request the next tool.
- **Python tools:** perform exact file reads, filters, and arithmetic.
- **Agent loop:** choose, execute, observe, then choose again or stop.
- **Final renderer:** show measured results and source paths.

Figure 22 - The complete path, once more



For “Have we seen unstable validation behavior before?”, the practical answer path is: search notes, inspect the two suggested run records, show their rise-and-fall loss sequences, and cite both kinds of source. The next time you read a trace, ask four questions: **Who chose this action? What file supplied this number? What does the score really mean? What claim is still unproven?**

> **You should now understand - checkpoint E:** (E1) Which part is the source of truth? (E2) Who writes the current CLI's displayed answer? (E3) Why is this not multi-agent? (E4) What would a vector database speed up?

Appendix A - Complete worked trace

This is the compact real sequence preserved in `docs/TRACE_EXAMPLE.md`, with short tool results for legibility. A later model run may choose different calls.

```
USER -> Have we seen unstable validation behavior before?
AGENT -> search_notes({"query":"unstable validation behavior"})
TOOL RESULT -> lr_01_seed2 score 0.468; baseline_gap_001 score 0.415
AGENT -> get_experiment({"run_id":"lr_01_seed2"})
TOOL RESULT -> epochs 3,4,5: 0.7136,0.8142,0.6793; final acc 76.35%
AGENT -> get_experiment({"run_id":"baseline_gap_001"})
TOOL RESULT -> epochs 1,2,3: 0.8696,1.4186,0.6663; final acc 71.25%
AGENT -> stop after 3 tool calls
PYTHON -> render inspected facts and source paths
```

The complete movement is: `app.py` receives question; `simple_agent.py` constructs system/user messages and sends them with available schema to Ollama; the model requests semantic retrieval; Python executes `search_notes`; MiniLM embeds the query and five observations; cosine sorts them; Python appends a `tool` result; the model asks for two exact record inspections in sequence; Python opens and validates each JSON; each result returns to Ollama; after it stops, `render_answer` prints only successful collected evidence.

The four underlying sources are `notes/lr_01_seed2.md`, `runs/lr_01_seed2.json`, `notes/baseline_gap_001.md`, and `runs/baseline_gap_001.json`. The note scores select candidates; the JSON curves establish the observed rises and falls. The saved answer does not establish *why* they occurred. In another session where Ollama stopped after search, the app's `APP POLICY` would visibly inspect the highest-ranked run before rendering.

Appendix B - Beginner checkpoint answers

A1. A shared validation set makes observed scores refer to the same held-out examples. **A2.** `model_seed` changes pseudo-random initial weights. **A3.** No: 128 is the batch size; all 10,000 selected images are traversed across 79 batches per epoch.

B1. `runs/*.json`, especially the `config` and per-epoch `metrics` fields. **B2.** A note paraphrases a run; it is not another measured training execution. **B3.** When the question specifies an exact recorded value such as learning rate 0.1.

C1. Vectors can rank related wording with few shared literal words. **C2.** A directional similarity score for that query and note, not a probability. **C3.** Semantic search retrieves candidates; RAG uses retrieved context in answer production. **C4.** It can mention another run or paraphrase a metric incorrectly; inspect primary JSON.

D1. Python, after checking the allowlist and arguments. **D2.** The exact run ID already identifies the file to read. **D3.** The model can request a next tool after observing a previous tool's result. **D4.** To prevent an endless or unproductive call chain.

E1. Saved run JSON for recorded settings and measurements. **E2.** Python's `render_answer`, using successful tool results. **E3.** Only one model decision loop is present. **E4.** Finding nearby vectors within a much larger collection without scanning every item.

Appendix C - How to run this checkout

In PowerShell, switch to the project and use the existing virtual environment directly:

```
Set-Location 'D:\Agentic_ai_rag projects'
.\.venv\Scripts\python.exe app.py
```

At the `>` prompt, try `What is the final validation accuracy of lr_01_seed3?`; type `exit` to quit. This direct JSON example was run successfully during PDF preparation and returned `71.45%` validation accuracy and `0.8109` validation loss at epoch 5. To ask once and exit:

```
.\.venv\Scripts\python.exe app.py --question "What is the final validation accuracy of lr_01_seed3?" --no-trace
```

Once the semantic dependency is repaired and the local Ollama server is running, you can type the behavior question “Have we seen unstable validation behavior before?” at the same interactive prompt. Its earlier successful command and full trace are preserved in `docs/TRACE_EXAMPLE.md`.

Local verification note (3 October 2026): during this PDF build, the current `.venv` could not import `sentence_transformers` because `typing_extensions` was missing, and `phase0.py --dry-run` could not import `torch`. Therefore the semantic trace was not freshly replayed in this session and training is not presented as a verified runnable command in this appendix. A repaired environment must install its required dependencies before those paths can run; no application source was changed here.

Appendix D - A project-specific study order

1. **Pass 1, concepts:** read Chapters 1-15 and Figures 1-11. Explain how images become saved run evidence and why notes need a different search method.
2. **Pass 2, retrieval:** read Chapters 16-19. Open `notes/lr_01_seed2.md` beside `runs/lr_01_seed2.json`; check the three loss values yourself.
3. **Pass 3, agent trace:** read Chapters 20-32 and Appendix A. Point to each model request, Python operation, and exact source. Remember the Python final renderer.
4. **Pass 4, code:** run the direct JSON question from Appendix C, then read `app.py`, `simple_agent.py`, `experiment_data.py`, and `note_retrieval.py` in that order. Use Chapter 35 as a map.
5. **Pass 5, independent explanation:** cover the guide and explain the unstable-validation question from user input to source-backed answer. Check the glossary only when you get stuck.
6. **Pass 6, small experiment after the environment is repaired:** ask the same question with different wording, watch whether the top note IDs or tool calls change, and compare the answer to JSON. Avoid changing training runs while learning this path.

Source map

Current application: `app.py`, `simple_agent.py`, `experiment_data.py`, `note_retrieval.py`, `requirements.txt`, `README.md`. Training laboratory: `phase0.py`. Saved evidence: `runs/*.json`, `notes/*.md`. Preserved trace and architecture: `docs/TRACE_EXAMPLE.md`, `docs/ARCHITECTURE.md`, `docs/LEARNING_GUIDE.md`. Historical learning path: `build_context.py`, `prepare_answer_prompt.py`, `generate_answer.py`, `evaluate_retrieval.py`, `retrieval_eval_cases.json`, `reports/lr_005_seed3_vs_lr_01_seed3.md`, `hypotheses/learning_rate_outcome_002.md`.